

For a Senior Delivery Consultant — Data Engineering (AWS Professional Services) first-round interview with a Sr. ProServe Cloud Architect, these are the questions I would expect, ranked by probability.

Section 1 — Tell me about yourself (100%)

1. Tell me about yourself.
2. Why AWS?
3. Why Professional Services?
4. Why are you leaving your current company?
5. What excites you about this role?

. . .

"I'm very proud of what my team and I have accomplished at Makro. In my current role, I set the enterprise data and AI strategy, built the team from 8 to 30 engineers, modernised our architecture into a lakehouse processing over 10 billion rows daily, and successfully deployed several core AI and GenAI initiatives.

Having successfully established these scalable data and AI foundations, I'm looking for my next challenge. Over my career across retail, e-commerce, and banking, I've realized that I am most energized when solving high-stakes architectural and transformation problems.

Joining AWS Professional Services allows me to take that battle-tested experience in enterprise cloud modernisations, large-scale data platforms, and MLOps, and apply it across a much broader set of enterprise clients on a global scale."

Key Pillars of This Strategy

- **Focus on Growth, Not Escape:** Frame your move as looking for new challenges and broader scale, not leaving a problem.
-
- **Show Complete Execution:** Highlight that you completed major foundation-building work at your current role (modernized architecture, scaled the organization, delivered ROI).
-
- **Connect to Professional Services:** Show that moving from an internal leadership role to client consulting at AWS ProServe is a deliberate strategic choice to broaden your impact.

1) Tell me about yourself.

- I'm a Data and AI leader with over 15 years of experience building large-scale cloud data platforms across e-commerce, retail, and banking in Southeast Asia.

Right now, I'm the Head of Data Engineering at Makro. I manage a 30-person team and an \$8B omnichannel business platform. We handle over 10 billion rows of data daily on cloud platforms and have built AI solutions for dynamic pricing, real-time analytics, and conversational AI.

Before this, I was VP of Data Engineering at Lazada, where I built real-time tracking engines for 100 million daily events, and worked in banking on large data pipelines.

I'm an AWS Certified Solutions Architect currently doing my Master's in Data Science at Northwestern. I focus on using AWS and cloud architecture to solve high-scale business problems, which is why I'm excited about this role."

3 Main Points to Remember

- **Who you are:** Senior Data/AI Leader with 15+ years experience.
- **What you've built:** 10 billion+ daily rows at Makro, 100 million+ daily real-time events at Lazada.
-
- **Your AWS link:** AWS Certified + experienced in multi-cloud architecture and MLOps.

2 .Why AWS?

| I want to work at AWS for three main reasons:

1. Scale & Innovation: AWS is the industry standard for cloud and AI infrastructure. In my experience building platforms processing billions of records, AWS has consistently provided the reliability and scale required for mission-critical workloads. Joining AWS lets me solve these infrastructure challenges at an unmatched global scale.

2. Alignment with Core Leadership Principles: I strongly align with AWS's focus on *Customer Obsession* and *Think Big*. My approach to engineering is always to work backward from the customer's business needs to build simple, resilient architecture.

3. Continuous Learning: AWS pushes the boundaries of cloud engineering and generative AI platforms. For me, it is the best place to keep growing alongside world-class engineers."

Core Pillars to Highlight

- **Tech Scale:** Building solutions that support high-volume data streaming and enterprise AI deployment.
-
- **Customer Obsession:** Designing cloud architecture focused strictly on business impact and ROI.
-
- **Growth:** Continuous learning in complex multi-region environments

AWS Professional Services (ProServe) is the natural progression for my career because it allows me to combine enterprise-level strategic leadership with deep technical execution to drive cloud transformations across multiple global client organizations.

1. Expanding Impact Beyond a Single Enterprise

- In my roles at Makro, Lazada, and SCB, I built and scaled enterprise platforms for individual organizations. ProServe allows me to leverage those battle-tested blueprints—such as Lakehouse modernizations, real-time streaming engines, and MLOps frameworks—to help dozens of enterprise clients innovate faster.
-

2. Bridging C-Suite Strategy with Deep Technical Execution

- Enterprise cloud adoption requires technical leaders who can guide CXOs through strategy, ROI, and organizational change while leading architectural delivery. Having managed 30-person organizations, multi-million-dollar budgets, and multi-cloud infrastructure, I naturally operate at the intersection of business strategy and technical architecture.
-

3. Accelerating Enterprise AI & Modernization

- Many enterprises struggle to modernize legacy data systems and safely deploy production Generative AI. I enjoy solving complex architectural challenges—like migrating legacy warehouses to cloud Lakehouses or building GenAI analytics platforms—to help enterprises achieve concrete, measurable business outcomes.
-

3. Why Professional Services?

“I’m very proud of what my team and I have accomplished at Makro. In my current role, I set the enterprise data and AI strategy, built the

team from 8 to 30 engineers, modernised our architecture into a lakehouse processing over 10 billion rows daily, and successfully deployed several core AI and GenAI initiatives.

Having successfully established these scalable data and AI foundations, I'm looking for my next challenge. Over my career across retail, e-commerce, and banking, I've realized that I am most energized when solving high-stakes architectural and transformation problems.

Joining AWS Professional Services allows me to take that battle-tested experience in enterprise cloud modernisations, large-scale data platforms, and MLOps, and apply it across a much broader set of enterprise clients on a global scale.”

Key Pillars of This Strategy

- **Focus on Growth, Not Escape:** Frame your move as looking for new challenges and broader scale, not leaving a problem.
-
- **Show Complete Execution:** Highlight that you completed major foundation-building work at your current role (modernized architecture, scaled the organization, delivered ROI).
-
- **Connect to Professional Services:** Show that moving from an internal leadership role to client consulting at AWS ProServe is a deliberate strategic choice to broaden your impact.
-

Interview Answer Script

“What excites me about the **Senior Delivery Consultant—Data Engineering** role in AWS Professional Services is the opportunity to take battle-tested enterprise data blueprints and help diverse organizations modernize their cloud architectures at scale.

Three key aspects of this role specifically stand out to me:

1. Solving Complex Enterprise Data Problems at Global Scale

- Throughout my career, I’ve architected data platforms processing over **10 billion daily records** at Makro and **100 million real-time events** at Lazada. What excites me about ProServe is applying these high-scale architectures—modern Lakehouses, real-time pipelines, and governance models—to solve critical data challenges across a variety of enterprise clients.

.

2. The Perfect Blend of Technical Leadership and Delivery Excellence

- A Senior Delivery Consultant doesn't just design architectures on paper; they ensure successful production delivery. Having managed **30-person engineering organizations, \$3M budgets,** and complex cloud modernizations, I thrive in roles where I can bridge C-suite business strategy with hands-on architectural execution.

-

3. Accelerating GenAI and Advanced Analytics on AWS

- Enterprise clients are looking to leverage Generative AI and MLOps, but they need solid cloud data foundations to do so safely. With my experience delivering production GenAI applications (such as Text-to-SQL platforms) and AWS architectural background, I'm excited to help AWS customers transform raw enterprise data into intelligent, real-time decision engines."

-

Core Talking Points to Emphasize

- **Hands-On Delivery + Consulting:** Highlight that you aren't just an advisor—you are a hands-on technical leader who drives architecture into production.

-

- **Multi-Cloud to AWS Standard:** Position your broad experience across cloud platforms (Databricks, multi-cloud) and AWS Solutions Architect background as an asset for clients migrating legacy workloads to AWS.
-
- **Business Impact:** Always tie data engineering back to tangible client outcomes (e.g., faster time-to-market, cost optimization, operational efficiency).
-

Section 2 — Leadership Principles (100%)

Customer Obsession

1. Tell me about a difficult customer.
2. Tell me about a customer who changed your approach.
3. Describe a time you went above and beyond for a customer.

Situation

At CP AXTRA (Makro), key business functions—Commercial, Operations, Finance, and Supply Chain—were using conflicting definitions for core metrics like sales, margin, and inventory across an \$8B omnichannel operation. Executive meetings frequently stalled because leaders debated whose figures were correct, which eroded trust in the reporting system.

Task

I led the enterprise KPI unification initiative to establish a single source of truth across a Databricks Lakehouse processing over 10 billion rows daily. The primary obstacle was alignment: resolving deep-seated operational friction among senior stakeholders who each believed their department's methodology was the correct one.

Action

- **Earned Trust via 1-on-1 Interviews:** Met individually with each domain head to understand how they calculated their metrics, the operational reasons behind their logic, and their core reporting priorities.
-
- **Facilitated Consensus Workshops:** Hosted structured cross-functional sessions focused on business process mapping rather than technical arguments, guiding teams to agree on shared business definitions.
-
- **Embedded Centralized Governance:** Documented calculation logic, data lineage, validation rules, and business owners in an enterprise KPI catalog, integrating these rules directly into our governed data pipeline.
-

- **Delivered Standardized Analytics:** Built a unified semantic layer and deployed 50+ executive dashboards to enforce consistent logic across all business units.

-

Result

- **Single Source of Truth:** Eliminated data discrepancies across executive reporting, moving leadership conversations from arguing over numbers to making strategic decisions.
-
- **Operational Impact:** Directly contributed to automating the month-end financial close cycle, reducing processing time from 5 days to 2 hours.
-
- **Cross-Functional Alignment:** Rebuilt trust between technology and business teams, creating a reliable foundation for downstream AI and demand-forecasting initiatives.
-

How to handle the follow-up: “*Why was the customer difficult?*”

“They weren’t difficult—they had competing, valid business priorities. Finance focused on auditability, Commercial focused on growth metrics, and Operations focused on execution. My job wasn’t to pick a winner, but to earn trust, align their priorities, and translate that consensus into a resilient cloud architecture.”

Situation

While rolling out an ML-driven demand forecasting and dynamic pricing initiative for 1,000+ vendors across 200+ stores at Makro, my engineering team built a complex gradient-boosting model to optimize inventory availability and margins. Although the model achieved high offline evaluation metrics, the Commercial and Merchandising teams refused to adopt the automated recommendations. They found the model to be a “black box” and manually overrode 80% of the output because it did not reflect their operational intuition.

Task

I had to fundamentally change my approach from focusing on algorithmic perfection to building customer trust, model explainability, and operational adoption.

Action

- **Shifted to Explainable AI:** Redesigned the model output architecture to expose key prediction drivers—such as seasonality, competitor price shifts, and vendor lead times—directly within the business interface.
-
- **Implemented Human-in-the-Loop Guardrails:** Created configurable business rules and override thresholds, allowing merchandisers to adjust parameters while feedforward mechanisms captured their feedback for model retraining.
-
- **Embedded Engineers with Business Teams:** Had senior engineers shadow merchandisers during daily stock and pricing reviews to directly observe how domain experts make real-time decisions.
-

Result

- **High Adoption & Performance:** Recommendation adoption rose from 20% to over 85%, directly improving store stock availability, procurement accuracy, and margin competitiveness across the catalog.
-
- **Faster Iteration Cycles:** Incorporating feedback loops reduced model variance and established a repeatable blueprint for deploying GenAI and predictive models across the organization.
-

What I Learned

This experience reshaped my philosophy as a technology leader. I learned that **Customer Obsession** means working backward from user trust and operational workflows—not just technical benchmarks. A simpler, explainable solution that users trust and adopt drives far greater business value than a complex architecture that sits unused.

Situation At CP AXTRA (Makro), we were building a Customer 360 initiative to unify customer records across store transactions, e-commerce, loyalty, and marketing for an \$8B retail business. The initial ticket assigned to my team was purely technical: integrate the raw data streams and build a unified customer dataset.

Task My formal scope was strictly to deliver the data platform pipelines. However, after speaking directly with commercial and marketing stakeholders, I realized that delivering a raw unified table wouldn't solve their real problem—they lacked the tools and structure to actually use that data for segmentation, campaign targeting, and executive decision-making.

Action

- **Engaged Business Users:** I expanded our scope to map business customer journeys, define campaign KPIs, and identify key marketing attributes needed for self-service analysis.
-
- **Built Identity Resolution & Semantic Layers:** Implemented deterministic and probabilistic identity resolution to unify online and offline user touchpoints, and built a governed semantic layer so non-technical users could query attributes effortlessly.
-
- **Enabled Campaign Activation:** Partnered with marketing ops to integrate the unified data foundation directly into campaign activation tools and MarTech automation workflows.
-

Result

- Transformed a static data pipe into a fully functional Customer Data Platform (CDP) actively used by business teams for personalized marketing campaigns.
-
- Created a reusable, governed customer foundation that later served as the baseline for downstream ML propensity models and GenAI profiling tools.
-

Key Takeaway

“Going above and beyond isn’t about working extra hours—it’s about looking past the technical ticket to understand the business outcome the customer actually needs. I didn’t stop when the technology was delivered; I stopped when the customer could actually realize the value.”



Technical Deep-Dive Preparedness

If the AWS interviewer asks “How did you handle this technically?”, use these concise responses:

- **Identity Resolution:** Combined deterministic matching (loyalty IDs, hashed phone numbers, emails) with probabilistic fuzzy matching using embedding-based similarity algorithms to deduplicate profiles across channels.
-
- **Architecture & Governance:** Built the CDP on a Databricks Lakehouse architecture, establishing governed data models (ODS/CDM/ADS layers) and automated data quality checks before exposing customer attributes to downstream MarTech tools.
-

Ownership

1. Tell me about a project you owned end to end.
2. Tell me about a time you stepped outside your role.
3. Tell me about a difficult migration.

1. Tell me about a project you owned end to end.

Use your AI document processing platform story.

“At CP AXTRA, I owned an AI document processing initiative end to end. The business was manually processing invoices, purchase orders, and delivery documents, which was slow, repetitive, and prone to errors.

I started by meeting Operations, Finance, and IT to understand what success meant for each group. Operations cared about extraction accuracy, Finance cared about validation and auditability, and the business cared about turnaround time.

I then defined the architecture and delivery plan. Documents were ingested securely, processed through OCR and AI extraction, validated using deterministic business rules, and low-confidence cases were routed to human review. I also made sure we had monitoring, audit logging, and downstream integration with finance and analytics systems.

I stayed involved from discovery through architecture, implementation, testing, stakeholder review, and rollout.

The project reduced manual effort and created a reusable automation pattern for other document workflows.

What I learned is that end-to-end ownership means not stopping at technical delivery. You own stakeholder alignment, risk, adoption, and the final business outcome.”

Leadership Principles: Ownership, Customer Obsession, Deliver Results, Invent and Simplify.

. . .

2. Tell me about a time you stepped outside your role.

Use your DBS Hadoop migration / user adoption story.

“At DBS, I was responsible primarily for the technical migration from a traditional data warehouse to a Hadoop-based environment.

Technically, the migration was progressing well, but as we approached go-live, I realized business users were not comfortable with the new environment. Training and change management were not really part of my formal responsibility, but I could see that the project could technically succeed and still fail from an adoption perspective.

I raised the risk with project leadership and stepped outside my core migration role. I worked directly with business users to identify critical workflows, organized hands-on training, created simple support material, identified power users, and helped establish hypercare support for go-live.

The transition became much smoother and users were better prepared to work on the new platform.

The lesson for me was that ownership is not about saying, ‘That’s outside my scope.’ If something threatens the customer outcome, I believe I need to help solve it.”

Leadership Principles: Ownership, Customer Obsession, Earn Trust.

. . .

3. Tell me about a difficult migration.

Use your **CP AXTRA AWS/Azure consolidation** story.

“At CP AXTRA, we had a fragmented data landscape. Some online workloads were running on AWS, while retail and analytics workloads were running on Azure and Databricks. This created duplicate technology stacks, additional cloud cost, context switching for engineers, and confusion around long-term architecture.

The difficult part was not moving data. Different stakeholders had different views on the target platform, and there were concerns about Power BI continuity, business downtime, migration risk, and cost.

I stepped in to structure the problem. Instead of asking, ‘Which cloud should we use?’, I reframed it around business outcomes: reduce duplication, lower operating cost, simplify engineering, improve governance, and protect business continuity.

I worked with engineering, BI, business, and platform stakeholders to compare migration options and document the trade-offs around cost, speed, complexity, and risk.

We then built a phased migration plan rather than taking a big-bang approach, with validation, parallel running, and rollback considerations built in.

The result was that the team moved from conflicting opinions to a clear migration direction with known risks and milestones.

What I learned is that difficult migrations are usually less about moving data and more about managing trade-offs, stakeholders, and business continuity.”

Leadership Principles: Dive Deep, Ownership, Have Backbone and Commit, Deliver Results.

For the interview, remember the three anchors:

AI documents = end-to-end ownership

DBS = stepped outside role

Cloud consolidation = difficult migration

1. End-to-End Ownership: AI Document Processing Platform

- **Target LPs:** Ownership, Customer Obsession, Deliver Results, Invent and Simplify
- PDF
-
-
- **Situation:** At CP AXTRA (Makro), manual processing of invoices, purchase orders, and delivery receipts across our omnichannel retail operations was slow, error-prone, and created severe operational bottlenecks for Finance and Logistics.
- PDF
-
-
- **Task:** I took full end-to-end ownership of designing, building, and scaling an automated AI Document Processing platform to replace manual data entry across operations.
- PDF
-
-
- **Action:**
-
- **Stakeholder Alignment:** Conducted discovery sessions with Operations, Finance, and IT to align on success criteria: extraction accuracy for Operations, auditability for Finance, and processing turnaround time for the business.
- PDF
-
-

- **Architecture & Delivery:** Architected a pipeline featuring secure document ingestion, OCR and LLM-based entity extraction, deterministic validation rules, and a automated human-in-the-loop exception queue for low-confidence outputs.
- PDF
-
-
- **Integration & Governance:** Built automated audit logging, monitoring, and direct integration into downstream ERP and analytics engines.
- PDF
-
-
-
- **Result:** Significantly reduced manual document processing effort, accelerated invoice processing cycles, and established a reusable cloud AI automation blueprint for future document workflows.
- PDF
-
-
- **Takeaway:** “End-to-end ownership doesn’t stop at technical delivery—you own stakeholder alignment, risk management, user adoption, and ultimate business ROI.”
- PDF
-
-

2. Stepping Outside Your Role: DBS Data Platform Adoption

- **Target LPs:** Ownership, Customer Obsession, Earn Trust
- PDF
-
-
- **Situation:** During a core data warehouse migration to a Hadoop enterprise platform at DBS, engineering progress was on schedule, but business analytics teams were struggling to transition to the new query environment.
- PDF
-
-
- **Task:** My formal role was strictly data pipeline engineering. However, I recognized that delivering a functioning platform that users couldn't operate would lead to project failure.
- PDF+ 1
-
-
- **Action:**
-
- **Identified Adoption Risks:** Flagged user readiness risks to leadership and voluntarily stepped outside my core technical boundaries.
- PDF
-
-
- **Hands-on Enablement:** Worked directly with business units to map daily query workflows, built practical training guides, and identified domain "power users" to drive peer-to-peer learning.

- PDF
-
-
- **Hypercare Support:** Established a dedicated hypercare channel during cutover to resolve user queries and technical friction in real time.
- PDF
-
-
-
- **Result:** Facilitated a seamless user transition to the new data environment with minimal operational impact and high post-migration user adoption.
- PDF
-
-
- **Takeaway:** “Ownership means never saying ‘that’s not my job.’ When a customer outcome or business goal is at risk, you step in and solve it.”
- PDF
-
-

3. Difficult Migration: Multi-Cloud Consolidation & Architecture Alignment

- **Target LPs:** Dive Deep, Ownership, Have Backbone; Disagree and Commit, Deliver Results
- PDF
-

-
- **Situation:** At CP AXTRA, online workloads ran on AWS while retail analytics ran on Azure and Databricks. This fragmented architecture led to duplicate stacks, escalating cloud infrastructure costs, engineering overhead, and debate over long-term strategy.
- PDF+ 1
-
-
- **Task:** I was tasked with navigating stakeholder friction and establishing a unified target cloud architecture without disrupting active retail operations across an \$8B business.
- PDF
-
-
- **Action:**
-
- **Reframed around Outcomes:** Shifted discussions away from cloud preferences toward concrete business criteria: operational cost reduction, governance simplicity, and business continuity.
- PDF
-
-
- **Deep-Dive Trade-off Analysis:** Partnered with engineering, BI, and business teams to audit workloads, documenting trade-offs across cost, latency, Power BI dependencies, and migration risks.
- PDF
-

-
- **Phased Migration Strategy:** Designed a phased execution plan with parallel runs, automated validation steps, and fallback mechanisms rather than a high-risk big-bang migration.
- PDF
-
-
-
- **Result:** Unified engineering and business stakeholders behind a clear, phased cloud architecture strategy, reducing duplicate operational overhead and setting a clear roadmap for enterprise execution.
- PDF
-
-
- **Takeaway:** “Difficult migrations are rarely just technical data movements—they are exercise in trade-off analysis, stakeholder management, and protecting business continuity.”
- PDF
-
-

Earn Trust

1. Tell me about influencing stakeholders without authority.
2. Tell me about a disagreement with business users.
3. How do you gain executive trust?

1. Influencing Stakeholders Without Authority (AI Document Processing)

- **Target Leadership Principles:** Earn Trust, Customer Obsession, Invent and Simplify
- PDF
- **Situation & Task:** At CP AXTRA (Makro), manual processing of invoices, purchase orders, and delivery documents created significant operational friction. I was tasked with leading an AI document automation initiative across Finance, Operations, Business, and IT. Because none of these cross-functional teams reported to me, I had to align competing priorities without direct positional authority.
- PDF+ 2
- **Action:**
- **Mapped Stakeholder Success Metrics:** Conducted discovery sessions to understand what each department prioritized— Operations needed extraction accuracy, the commercial team cared about turnaround time, Finance required auditability, and IT prioritized security and integration.
- PDF
- **Demonstrated Transparent POCs:** Built a working proof-of-concept using representative documents rather than pushing a theoretical production model. Transparently demonstrated both high-confidence extractions and failure modes.
- PDF+ 1

- **Designed Risk Mitigation:** Incorporated deterministic business rules and a human-in-the-loop review workflow for low-confidence extractions, giving stakeholders full visibility and control.
- PDF
- **Result:** Secured complete buy-in across all four departments, leading to a successful rollout that reduced manual document processing effort and established a scalable AI automation blueprint for future workflows.
- PDF
- **Takeaway:** “Influence without authority comes from deeply understanding stakeholder priorities, making operational risks visible, and proving value through low-risk prototyping.”
- PDF

2. Disagreement with Business Users (KPI Unification)

- **Target Leadership Principles:** Have Backbone; Disagree and Commit, Earn Trust, Dive Deep
- PDF
- **Situation & Task:** At CP AXTRA, business functions were using conflicting definitions for fundamental metrics like sales, inventory, and margin across an \$8B enterprise. Finance, Commercial, and Operations each insisted their calculation was correct, leading to prolonged debates in executive meetings over whose numbers were right.
- PDF+ 1
- **Action:**
- **Anchored on Ground Truth:** Met individually with each domain leader to understand the operational reasoning behind their calculations rather than forcing a top-down decision.
- PDF

- **Facilitated Process-Driven Workshops:** Hosted cross-functional alignment sessions focused on underlying business processes, data lineage, and edge cases rather than personal preferences.
- PDF
- **Codified Centralized Governance:** Agreed on single business definitions, assigned functional data owners, documented validation rules, and built the unified logic directly into a centralized semantic layer on our cloud platform.
- PDF
- **Result:** Replaced reporting discrepancies with a single source of truth across 50+ executive dashboards, shifting executive discussions from debating data accuracy to making strategic decisions.
- PDF+ 1
- **Takeaway:** “Disagreement shouldn’t be avoided—when you ground conversations in business processes, facts, and shared outcomes, constructive friction leads to a far more resilient solution.”

3. Gaining Executive Trust (Framework & Cloud Consolidation)

- **Target Leadership Principles:** Earn Trust, Dive Deep, Customer Obsession
- **Core Approach:**
-
- **Translate Tech into Business Outcomes:** Begin executive discussions with business objectives, risk mitigation, operational agility, and ROI rather than underlying technology choices.
- PDF
- **Provide Radical Transparency:** Highlight technical risks, cost trade-offs, and uncertainties early. Executives trust leaders who

present objective trade-offs over those who simply say what leadership wants to hear.

- PDF+ 1
- **Deliver Consistently with Visibility:** Establish predictable delivery cadence, provide clear progress metrics, and escalate blockers early when scope or assumptions shift.
- PDF
- **Concrete Example (Multi-Cloud Consolidation):**
- During architecture consolidation discussions between AWS and Azure workloads at CP AXTRA, stakeholders held strong, conflicting platform preferences.
- PDF
- Instead of debating cloud vendors, I reframed the discussion around business continuity, migration risk, engineering complexity, and operating costs.
- PDF
- By presenting a clear trade-off matrix and proposing a phased execution roadmap, leadership gained full clarity and confidently moved from debate to an aligned strategic direction.
- PDF

Dive Deep

1. Tell me about a production issue.

2. Describe a root cause analysis you performed.
3. Tell me about debugging a complex data issue.

The **M-H-E-R-M** framework (Metric $\rightarrow$ Hypotheses $\rightarrow$ Evidence $\rightarrow$ Root Cause $\rightarrow$ Mechanism) elevates a technical debugging story into an L7 executive demonstration of systemic ownership.

1. Tell Me About a Production Issue (Systemic Incident Response)

- **Metric / Signal:** Downstream analytics consumers reported missing transaction events in our real-time streaming pipeline. System-level CPU, memory, and broker metrics were operating within normal thresholds, masking the underlying failure.
-
- **Hypotheses:** Formulated four potential failure vectors: downstream ingestion failure, consumer group deserialization errors, network packet drops, or upstream payload rejection.
-
- **Evidence:** Extracted error logs across producers and brokers and correlated them with payload sizes. The failures occurred exclusively on records exceeding standard message limits, pinpointing the issue to the producer-broker boundary.
-
- **Root Cause:** An unannounced upstream schema update added new fields to transaction records, causing peak payload sizes to silently exceed the broker's configured maximum message threshold.
-
- **Mechanism (L7 Execution):** Immediate containment involved adjusting message thresholds and replaying the dead-letter

queue. To eliminate this entire class of failure across the enterprise, I established a mandatory schema-contract validation stage in our CI/CD pipelines, automated payload boundary monitoring, and deployed automated source-to-target reconciliation tools.

•

2. Describe a Root Cause Analysis You Performed (Scientific Method)

- **Metric / Signal:** Intermittent data dropouts affecting less than 2% of real-time transactions. Overall pipeline throughput and broker health metrics registered green.
-
- **Hypotheses:** Instead of tweaking configuration parameters blindly, I tested five hypotheses: (1) broker GC pauses, (2) consumer rebalancing, (3) network partitioning, (4) schema mismatch, and (5) payload size threshold breach.
-
- **Evidence:** Systematic log analysis revealed zero consumer lag spikes or network drops. Segmenting failed vs. successful messages by payload byte size showed a 100% correlation between dropped records and payloads exceeding default buffer limits.
-
- **Root Cause:** An upstream feature deployment expanded transaction payload size for specific enterprise accounts, breaching the message limit without triggering standard broker health alarms.
-
- **Mechanism (L7 Execution):** Validated the diagnosis via synthetic test payloads, updated the platform configuration, and replayed missing transactions. To prevent recurrence across all streaming teams, I implemented automated schema

contract testing during PR builds, pre-ingestion payload size validation, and proactive threshold alerting.

•

3. Debugging a Complex Data Issue (Data Integrity & Reconciliation)

- **Metric / Signal:** Downstream executive reporting showed a subtle count discrepancy compared to core transactional source databases.

•

- **Hypotheses:** Investigated three data integrity failure points: non-deterministic transformations in processing, silent consumer offset skips, or producer dropping unacknowledged records.

•

- **Evidence:** Audited record counts at every pipeline hop—Source DB $\rightarrow$ Producer $\rightarrow$ Kafka Topic $\rightarrow$ Consumer $\rightarrow$ Target Store. Discrepancies emerged strictly at the Producer-to-Broker publish phase for specific high-volume transaction types.

•

- **Root Cause:** Upstream schema changes expanded specific payload sizes beyond Kafka's maximum allowable record size, causing silent publish rejections for larger transactions.

•

- **Mechanism (L7 Execution):** Recovered lost events via point-in-time state replay. To secure enterprise data integrity long-term, I architected an automated end-to-end data reconciliation framework that compares source vs. target record signatures in real time and flags discrepancies before downstream reporting is impacted.

•

L6 vs. L7 Phrasing Reference

Metric Level	L6 Phrasing (Standard Execution)	L7 Phrasing (Systemic Mechanism)
Fixing the Bug	"I updated the Kafka message size configuration and replayed the missing messages."	"We remediated the immediate data gap via DLQ replay and aligned broker configurations."
Monitoring	"I added an alert for large message sizes in Kafka."	"I established real-time payload observability standards across all streaming ingestion pipelines."
Prevention	"I told the upstream team to notify us before changing schemas."	"I introduced automated schema-contract checks in CI/CD to prevent unannounced schema mutations from reaching production."

. Production Issue: Systemic Incident Response

- **Metric / Signal:** Downstream analytics dashboards stopped receiving a subset of real-time transaction events. Core platform health indicators—CPU utilization, memory usage, broker status, and consumer lag—were operating within normal thresholds, masking the underlying failure.
-
- **Hypotheses:** Formulated four potential failure vectors: (1) downstream consumer deserialization failure, (2) consumer group rebalance drops, (3) network packet loss, or (4) producer-side payload rejection.
-
- **Evidence:** Extracted error logs across producers and brokers, correlating timestamps with payload byte sizes. Dropped records occurred exclusively on messages with byte sizes significantly higher than average.
-
- **Root Cause:** An upstream deployment added new metadata fields to transaction events, causing peak payload sizes to

silently exceed Kafka's configured maximum message threshold (`message.max.bytes`).

-
- **Mechanism (L7 Execution):** Contained the immediate impact by updating configuration limits across producers, brokers, and consumers, then replaying missing records from dead-letter queues (DLQ). To eliminate this entire class of failure across the enterprise, I established **mandatory schema-contract validation within CI/CD pipelines**, automated payload-size observability standards across streaming pipelines, and created standardized alerting for payload growth anomalies.
-

2. Root Cause Analysis: Scientific Hypothesis Testing

- **Metric / Signal:** Intermittent data dropouts affecting roughly 1.5% of real-time records in a high-volume streaming engine processing 100M+ daily events. Overall pipeline throughput metrics registered green.
-
- **Hypotheses:** Avoided random configuration changes and systematically tested five hypotheses: (1) broker GC pauses, (2) consumer rebalance timeouts, (3) network partitioning, (4) schema mismatch, and (5) payload threshold breaches.
-
- **Evidence:** Analyzed logs across all pipeline hops. Zero network partition events or consumer lag spikes were detected. However, segmenting failed vs. successful records by payload size showed a 100% correlation between dropped messages and payloads exceeding buffer limits.
-
- **Root Cause:** An upstream feature update expanded transaction payloads for enterprise account profiles, breaching message size limits without triggering standard system health alarms.

-
- **Mechanism (L7 Execution):** Reproduced the failure using synthetic payloads, aligned parameters, and restored lost records via replay. To prevent recurrence platform-wide, I introduced **producer-side pre-ingestion contract validation**, integrated automated schema registry checks into pull requests, and set proactive threshold monitoring across all engineering domain squads.
-

3. Complex Data Issue: End-to-End Data Integrity

- **Metric / Signal:** Business stakeholders reported intermittent count discrepancies between upstream transactional databases and downstream reporting layers. The issue was intermittent rather than a complete outage.
-
- **Hypotheses:** Investigated three data integrity failure points: (1) non-deterministic transformations, (2) silent consumer offset skips, or (3) producer publish rejections.
-
- **Evidence:** Audited record counts systematically at every pipeline hop: Source DB $\rightarrow$ Producer $\rightarrow$ Kafka Topic $\rightarrow$ Consumer $\rightarrow$ Target Store. Discrepancies emerged strictly at the Producer-to-Broker boundary for large transaction records.
-
- **Root Cause:** Unannounced upstream schema payload growth caused specific high-volume transaction records to exceed Kafka buffer thresholds, leading to silent publish rejections.
-
- **Mechanism (L7 Execution):** Restored data parity by executing state replays. To secure enterprise data integrity long-term, I architected an **automated source-to-target data reconciliation**

framework that continuously compares source record signatures against target stores in near real time, isolating discrepancies before downstream executive dashboards are impacted.

•

Invent & Simplify

1. Tell me about automating a manual process.
2. Tell me about simplifying an architecture.

What was complex/manual → what insight you had → how you simplified it → what mechanism you created → measurable outcome.

1. Tell me about automating a manual process

Use your **AI document processing** story.

“At CP AXTRA, Finance and Operations teams were manually processing invoices, purchase orders, and delivery documents. Employees had to read scanned PDFs, extract supplier details, PO numbers, quantities, tax, totals, and then validate them before posting downstream.

The process was repetitive, slow, and vulnerable to human error.

•

My task was to automate it, but I didn't want to simply replace people with OCR. I first broke the process into clear stages: document ingestion, extraction, validation, exception handling, and downstream integration.

We used OCR and AI to extract structured fields, then deterministic business rules to validate things like whether the PO existed, whether the supplier matched, whether totals reconciled, and whether the invoice was duplicated.

High-confidence cases moved through automatically. Low-confidence or rule-failing cases were routed to human review.

I also made the workflow event-driven so each stage could scale independently and downstream systems were loosely coupled.

The result was a much faster turnaround time, reduced manual effort, and a reusable pattern that could be extended to other document types.

The key lesson for me was that good automation is not about removing humans completely. It is about automating predictable work and using people where judgment is actually needed."

L7 upgrade

Add:

“I also turned the solution into a reusable platform pattern rather than a one-off workflow, so future document use cases could reuse the same ingestion, validation, confidence, and human-review mechanisms.”

That makes it sound much more senior.

. . .

2. Tell me about simplifying an architecture

Use your **cloud/data platform consolidation** story.

“At CP AXTRA, our data landscape had become fragmented. Some workloads were running on AWS, others on Azure and Databricks, and different teams had built overlapping ingestion, transformation, governance, and reporting patterns.

The immediate problem was higher cost, but the deeper problem was operational complexity. Engineers had to maintain multiple stacks, switch context frequently, and business users were not always clear which platform was the source of truth.

I stepped back and reframed the problem from ‘which cloud should we use?’ to ‘what is the simplest architecture that meets our business requirements?’

I mapped the duplicate capabilities, identified which workloads truly needed separate platforms, and where we could standardize common patterns for storage, transformation, governance, and analytics.

Rather than performing a risky big-bang migration, I proposed a phased consolidation plan with clear migration waves, validation, business continuity, and rollback options.

The result was a simpler target architecture, clearer ownership, reduced duplication, lower operating complexity, and a path toward lower platform cost.

What I learned is that simplification is not removing technology for the sake of it. It is reducing unnecessary choices and operational burden while preserving the capabilities the business actually needs.”

Strong L7 closing

“At senior level, I think simplification is about creating mechanisms and standards that prevent complexity from growing back.”

That line is very Amazon-style.

For memory, use:

Automation = Documents

Simplification = Cloud/platform consolidation

1. Automating a Manual Process: AI Document Processing Platform

- **What was Complex / Manual:** At CP AXTRA (Makro), Finance and Operations teams manually processed thousands of invoices, purchase orders, and delivery documents across our retail network. Employees had to read scanned PDFs, manually extract supplier details, PO numbers, tax totals, and line items, and reconcile them against ERP systems—a process that was slow, error-prone, and created operational bottlenecks.
- PDF+ 1
-
-
- **What Insight You Had:** Effective automation is not simply replacing humans with OCR or attempting 100% automated extraction. The key insight was decomposing the process into distinct, decoupled stages—ingestion, extraction, deterministic validation, exception routing, and downstream integration—so predictable work is automated while human judgment is reserved exclusively for low-confidence edge cases.
- PDF
-
-
- **How You Simplified It:** Architected an event-driven pipeline that used OCR and AI extraction to capture structured fields, followed by deterministic business rules to validate PO existence, supplier records, and arithmetic totals. High-confidence records passed through automatically to ERP systems, while failed validations or low-confidence extractions were automatically routed to a human-in-the-loop exception review interface.

- PDF+ 1
-
-
- **What Mechanism You Created (L7 Platform Pattern):** Turned the initial workflow into a standardized, reusable enterprise document processing pattern. Built modular ingestion APIs, generic schema validation rules, confidence-scoring engines, and reusable human-review interfaces so future business units could onboard new document types (e.g., customs declarations, vendor contracts) without re-architecting the core engine.
- PDF+ 1
-
-
- **Measurable Outcome:** Drastically reduced manual document processing turnaround times, cut operational processing effort, and eliminated manual data entry errors across core finance operations.
- PDF
-
-

2. Simplifying an Architecture: Cloud & Data Platform Consolidation

- **What was Complex / Manual:** CP AXTRA's data environment had become heavily fragmented. Workloads were split across AWS, Azure, Databricks, and legacy warehouses, with different teams building redundant pipelines, governance models, and BI reports. This fragmentation increased operational costs, caused

heavy context-switching for our 30-person engineering team, and created confusion over which system served as the single source of truth.

- PDF+ 2
-
-
- **What Insight You Had:** Simplification is not about removing technology for its own sake; it is about reducing operational burden and eliminating unnecessary architectural choices while preserving business capabilities. I reframed the discussion from “which cloud platform is better” to “what is the simplest architecture that meets our business requirements across an \$8B enterprise?”
- PDF+ 1
-
-
- **How You Simplified It:** Conducted a comprehensive audit of duplicate capabilities, mapped pipeline workloads, and defined a unified multi-layer Lakehouse target architecture on Databricks processing 10B+ daily records. Instead of a high-risk big-bang migration, I executed a phased consolidation plan with clear migration waves, automated validation gates, parallel runs, and rollback strategies.
- PDF+ 1
-
-
- **What Mechanism You Created (L7 Platform Governance):** Created an **Architecture Review Board (ARB)** along with standardized, reusable data pipeline blueprints and automated CI/CD schema validation gates. This permanent mechanism ensures that any new data initiative adheres to the unified architecture, preventing technical debt and stack fragmentation from growing back as the organization scales.

- PDF
-
-
- **Measurable Outcome:** Successfully consolidated fragmented multi-cloud workloads into a single enterprise Lakehouse processing 10B+ daily rows, eliminated redundant infrastructure costs, reduced pipeline maintenance overhead, and unified reporting across 50+ executive dashboards.
- PDF+ 1
-
-

Deliver Results

1. Biggest project.
2. Biggest impact.
3. Biggest failure.
4. Missed deadline.

For **Deliver Results**, Amazon L6/L7 answers should show:

Goal → constraints → what you personally drove → measurable outcome → mechanism/lesson.

Use four different stories so you don't sound repetitive.

1. Biggest project

Use your **cloud/data platform consolidation at CP AXTRA**.

“One of the largest programs I led was consolidating a fragmented data landscape across AWS, Azure, Databricks, BI, and multiple business domains.

The problem was not just cloud cost. We had duplicate engineering patterns, multiple governance models, context switching for engineers, and unclear long-term ownership.

My task was to create a target architecture and migration path without disrupting business reporting or ongoing operations.

I started by mapping workloads, dependencies, cost, user journeys, and business-critical reports. I then aligned engineering, BI, platform, and business stakeholders around common objectives: reduce duplication, simplify operations, protect continuity, and improve long-term maintainability.

Rather than proposing a big-bang migration, I created phased migration waves with validation, rollback plans, and clear ownership.

The program reduced platform duplication, created a much clearer operating model, and established a path toward roughly halving parts of our cloud/platform cost.

The biggest lesson was that large transformation programs succeed when architecture, economics, and change management are treated as one problem rather than three separate ones.”

LPs: Deliver Results, Ownership, Think Big, Dive Deep.

. . .

2. Biggest impact

Use your **cost optimization** / **platform efficiency** story.

“One of my biggest measurable impacts was around data and BI platform cost optimization.

We had a large analytics environment with duplicate datasets, oversized models, inefficient refresh schedules, and underused assets. Monthly platform cost had grown significantly.

I led a detailed usage and cost analysis across datasets, refresh patterns, compute utilization, and report adoption.

Based on the findings, we consolidated duplicate semantic models, introduced incremental refresh, removed unused assets, improved workspace governance, and aligned capacity to actual demand.

This created a roadmap to reduce monthly cost by close to 50% while maintaining business-critical analytics performance and reliability.

What made the impact important was that we did not simply cut capacity. We changed the operating model so cost efficiency became part of how the platform was managed going forward.”

L7 line

“The result was not just savings; we created a repeatable FinOps mechanism so the cost would not grow back in the same way.”

. . .

3. Biggest failure

Use your **DBS Hadoop migration / adoption** story.

“At DBS, I was involved in a large migration from a traditional data warehouse to a Hadoop-based platform.

Technically, the migration was successful. We completed the major migration activities and system performance was acceptable.

But after go-live, adoption was slower than expected.

Looking back, I realized I had focused too heavily on technical delivery and not enough on user readiness, training, and change management.

Once I recognized that gap, I worked with business users to identify critical workflows, organized hands-on training, created support material, involved power users, and helped establish hypercare support.

Adoption improved, but I still consider it a failure in the sense that I should have identified that risk much earlier.

Since then, I never define success only as ‘the platform went live.’ I include adoption, training, stakeholder readiness, and business usage as formal delivery metrics from the beginning.”

That is a strong failure answer because you **own the mistake**.

Do not say:

“The users were not ready.”

Say:

“I underestimated user readiness.”

Big difference.

. . .

4. Missed deadline

For this one, use a project where requirements or dependencies changed. If you have a real example, use that. A good structure is:

“On one data platform initiative, we committed to a delivery date based on an initial set of requirements. During implementation, we discovered additional upstream dependencies and data-quality issues that had not been visible during planning.

I realized that meeting the original date would require either reducing quality or creating operational risk.

I took responsibility for the miss rather than pushing the team to deliver something unstable.

I immediately informed stakeholders, explained the specific dependencies and trade-offs, and proposed a revised plan with clear milestones.

I also separated must-have scope from lower-priority features so we could deliver the most valuable capability first.

We missed the original full-scope date, but delivered the critical business functionality earlier through a phased release and completed the remaining scope afterward.

The main lesson was that missing a date is sometimes unavoidable, but surprising stakeholders is not. Since then, I track external dependencies and risk indicators much earlier and communicate them before they become schedule issues.”

Amazon likes this because you show:

- No hiding bad news.
- Customer/business prioritization.
- Ownership.
- Trade-off judgment.
- Mechanism to prevent recurrence.

. . .

Easy memory mapping

Remember just four anchors:

Biggest project → Cloud consolidation

Biggest impact → Cost optimization

Biggest failure → DBS adoption

Missed deadline → Dependency / phased delivery

And for every Deliver Results answer, finish with:

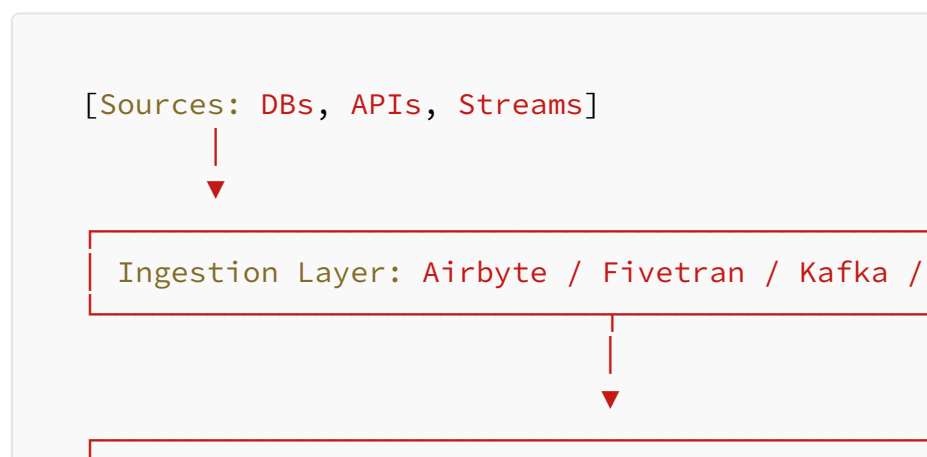
“I didn’t only fix the immediate problem; I changed the mechanism so we could deliver more predictably going forward.”

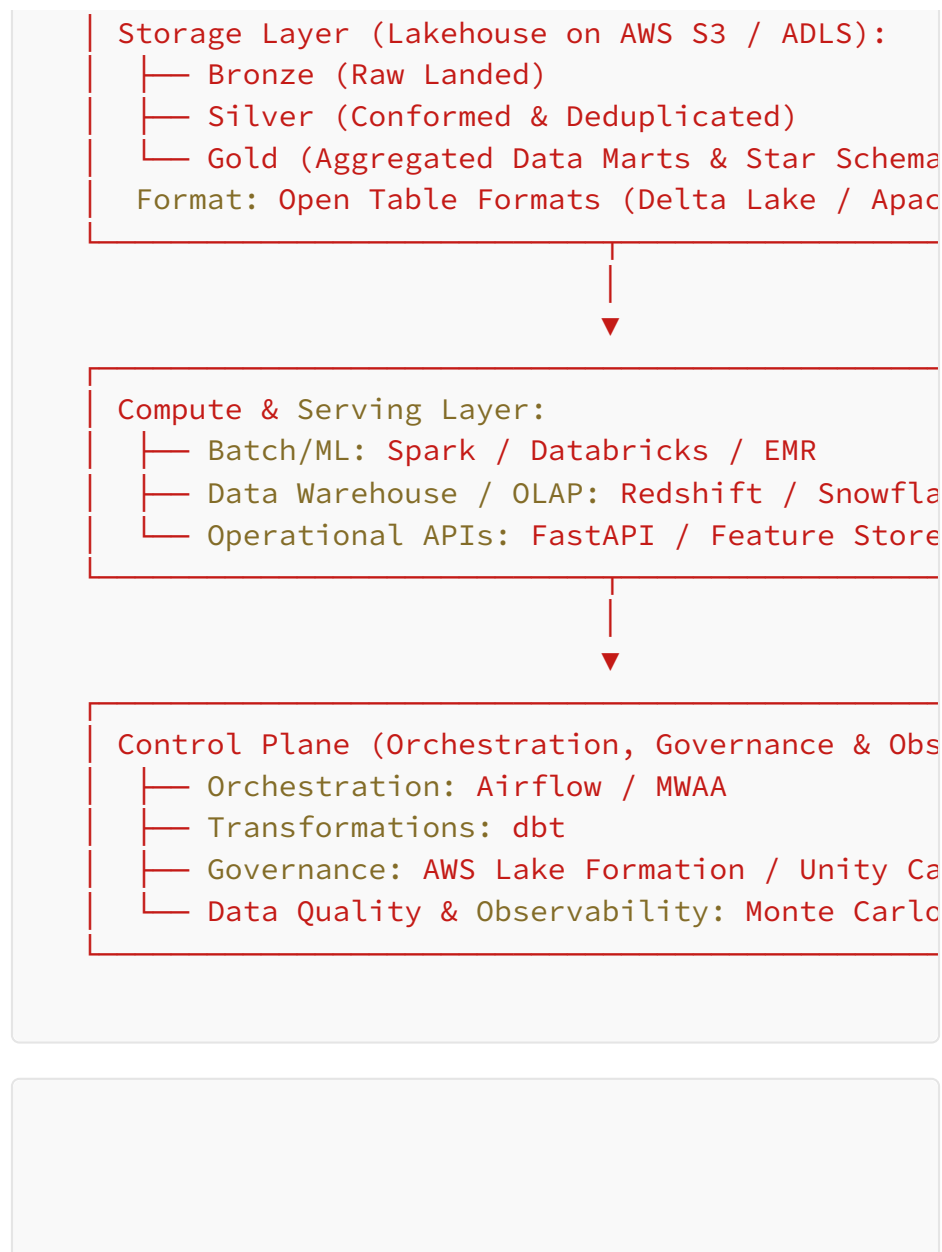
That makes the answer sound much more L6/L7 than purely operational.

Section 3 — Architecture (95%)

1. Design a modern data platform.
2. Design Customer 360.
3. Design a document processing platform.
4. Design a real-time analytics platform.
5. Design a streaming architecture.
6. Design a lakehouse.
7. Design a data governance platform.

1. Modern Data Platform Architecture

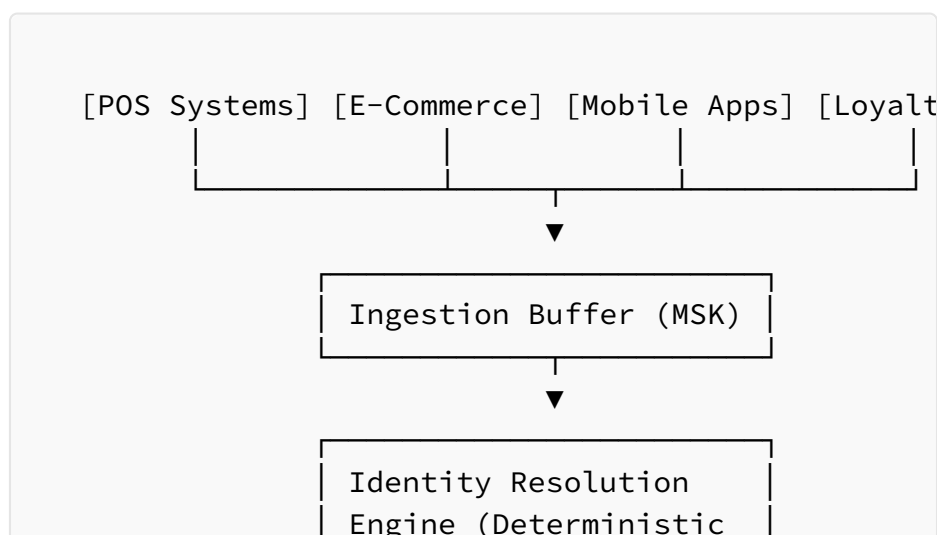


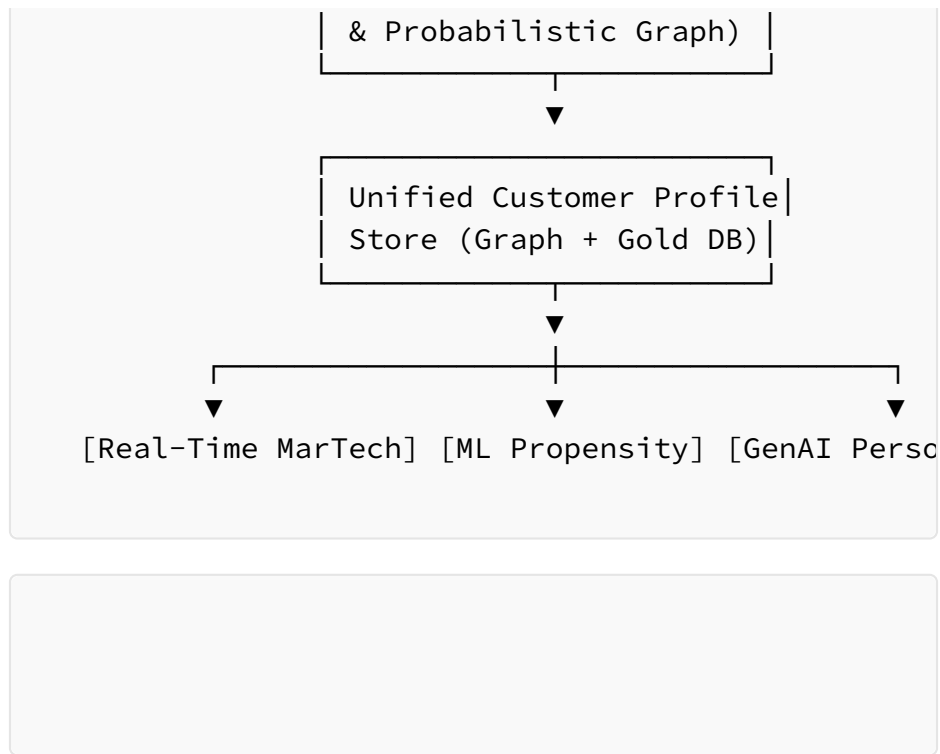


- **Ingestion Layer:** Hybrid batch (Fivetran/Airbyte) and streaming (Kafka/Debezium CDC) pipelines to capture operational data with minimal source latency.
-
- **Storage & Table Format:** Cloud object storage (S3/ADLS) using open table formats (Delta Lake or Apache Iceberg) to provide ACID transactions, schema enforcement, and time-travel querying.
-
- **Medallion Pipeline Architecture:**

- **Bronze:** Raw, immutable append-only ingestion.
-
- **Silver:** Cleaned, deduplicated, and conformed data models.
-
- **Gold:** Business-aggregated star schemas optimized for BI and downstream ML.
-
-
-
- **Serving & Transformation:** dbt for modular SQL transformations; Databricks/EMR for heavy Spark processing; Redshift/Snowflake for low-latency BI queries.
-
- **Control Plane:** Airflow for DAG orchestration, Apache Ranger/Unity Catalog for fine-grained access control, and Monte Carlo for automated pipeline observability.
-

2. Customer 360 Architecture

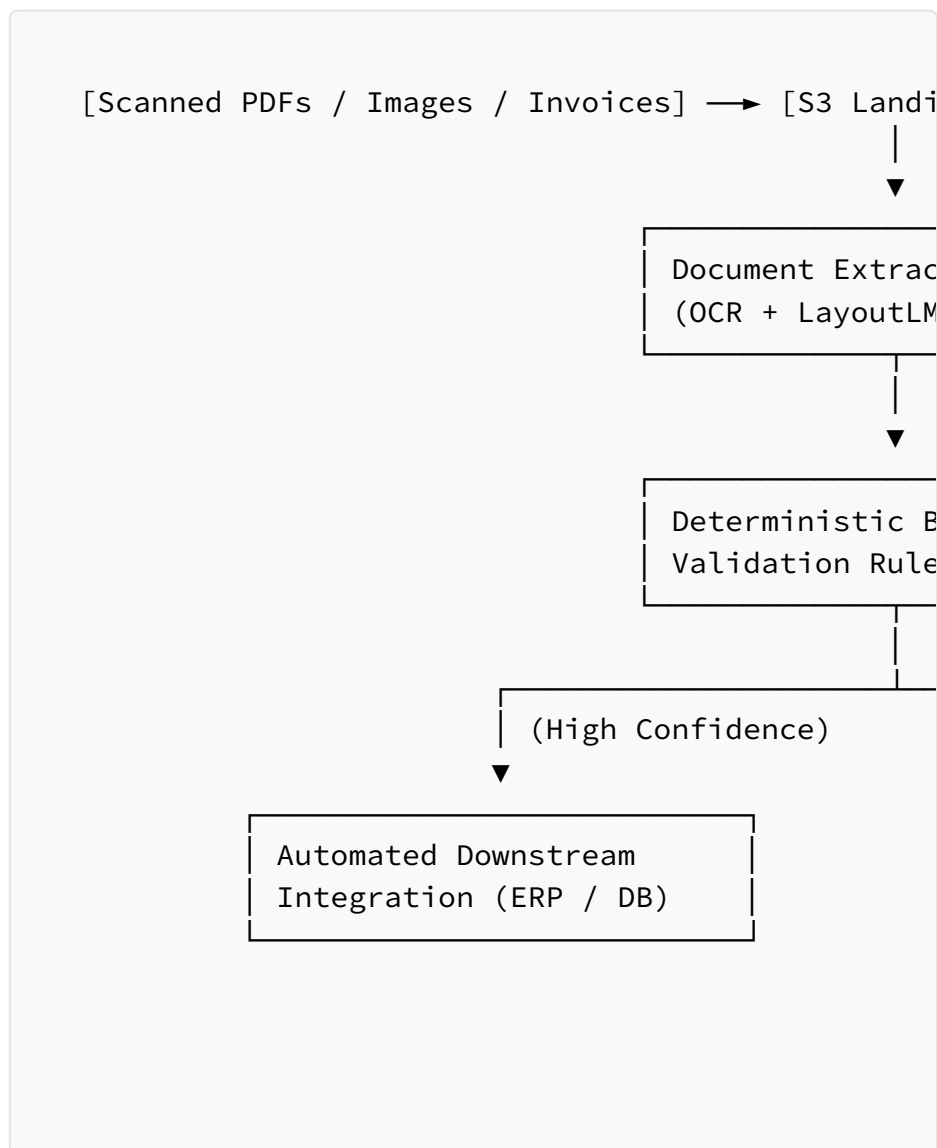




- **Multi-Channel Ingestion:** Stream real-time telemetry (clickstream, POS transactions) and batch updates (CRM, support tickets) into a central broker.
-
- **Identity Resolution Engine:**
- **Deterministic Matching:** Merges records based on exact matches (Hashed Email, Phone, Loyalty ID).
-
- **Probabilistic Matching:** Employs ML fuzzy matching and graph databases (Amazon Neptune) to link profiles based on shared device IDs, IPs, and address history.
-
-
-
-
- **Unified Profile Store:** Dual-tier storage featuring a high-speed KV store (DynamoDB/Redis) for real-time customer lookups and a Lakehouse Gold table for historical behavioral analytics.

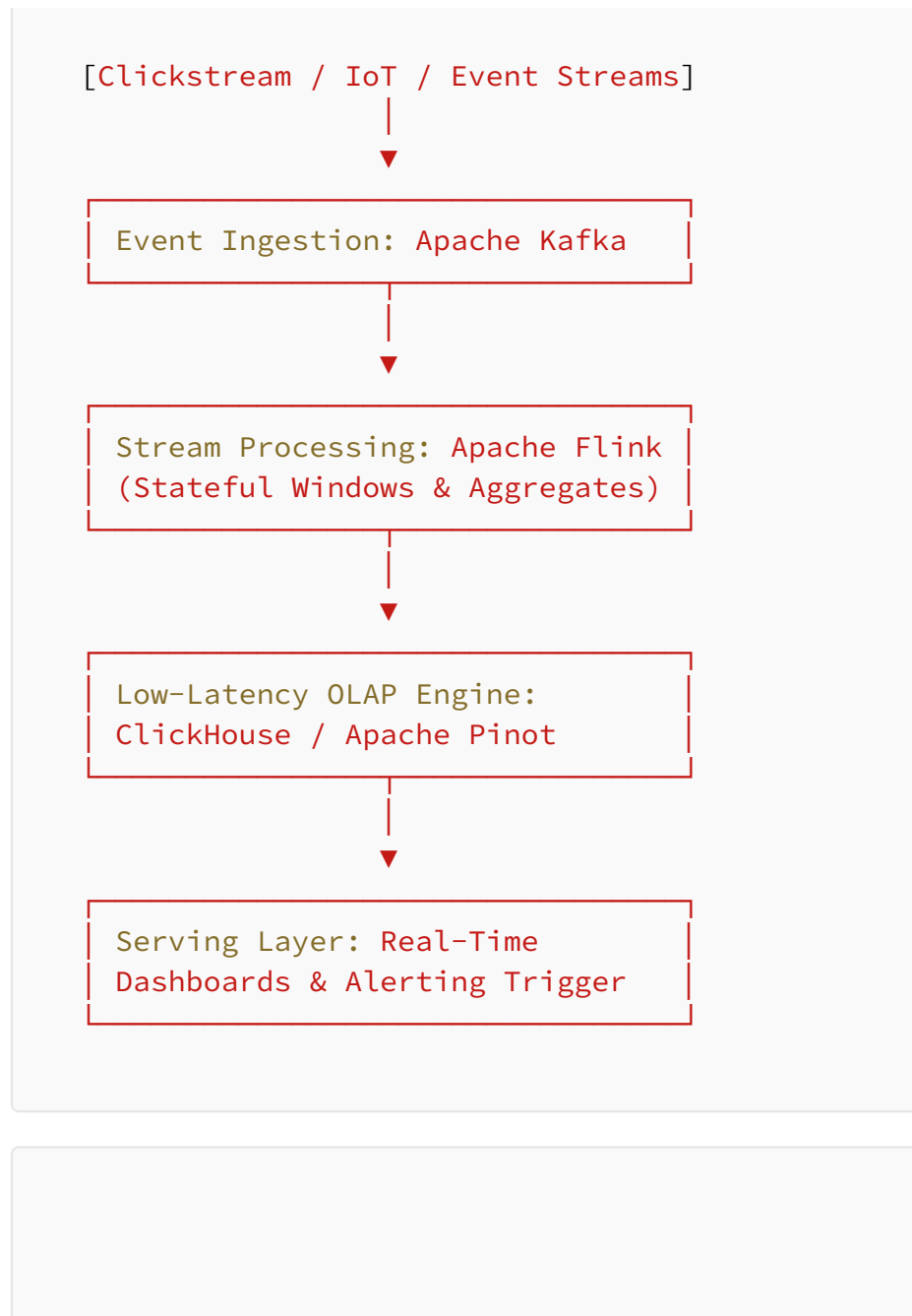
-
- **Activation Layer:** Exposes unified customer APIs directly to MarTech automation platforms (Braze/Salesforce), ML feature stores (churn/propensity scoring), and GenAI profile generators.
-

3. AI Document Processing Platform Architecture



- **Multi-Modal Ingestion:** Event-driven S3 ingestion triggered via Lambda/EventBridge upon document upload (PDFs, TIFFs, JPGs).
-
- **Extraction Engine:** Combines OCR (AWS Textract) for spatial layout extraction with specialized LLMs/LayoutLM models to output structured JSON data (e.g., PO numbers, line items, tax totals).
-
- **Validation & Rule Engine:** Runs deterministic logic to cross-check extracted data against ERP records (e.g., verifying PO presence, vendor match, arithmetic total balance).
-
- **Human-in-the-Loop (HITL) Exception Handler:** Routes documents with extraction confidence $< 85\%$ or failed validation checks to a custom web UI for operator verification; operator edits feed back into the retraining pipeline.
-
- **Audit & Storage:** Stores raw documents, extraction JSONs, and complete operator audit trails in object storage while syncing validated line items to downstream SAP/Oracle ERPs.
-

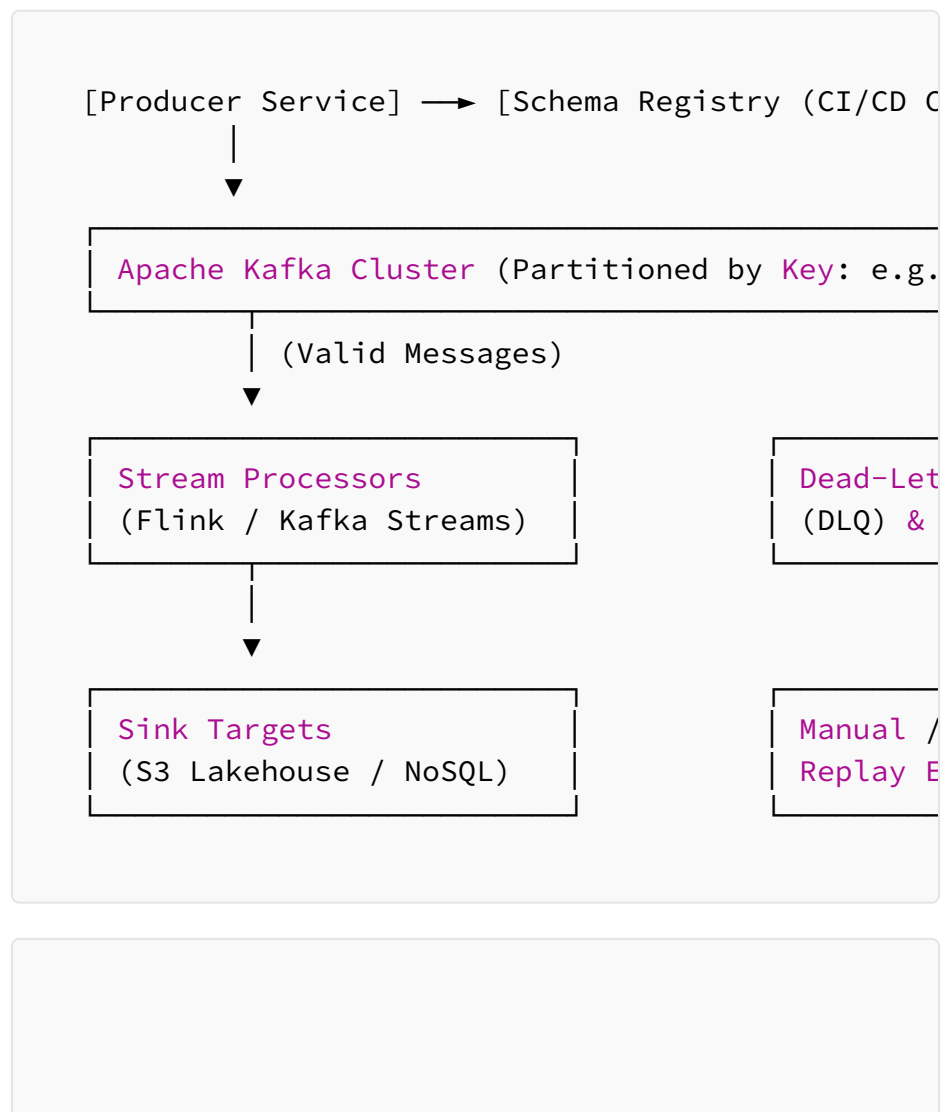
4. Real-Time Analytics Platform Architecture



- **Event Ingestion:** Highly partitioned Apache Kafka or Amazon MSK cluster accepting sub-second payload writes from distributed edge producers.
-
- **Stream Processing:** Apache Flink or Spark Structured Streaming executing sliding-window aggregations, anomaly detection, and stateful joins.
-

- **Low-Latency Storage Engine:** Columnar OLAP engines (ClickHouse, Apache Pinot, or Druid) designed for sub-second query performance over billions of rows.
-
- **Serving & Alerting:** Real-time push dashboards (Grafana/Superset) combined with automated alert dispatchers for threshold breaches.
-

5. Streaming Architecture (Event-Driven)

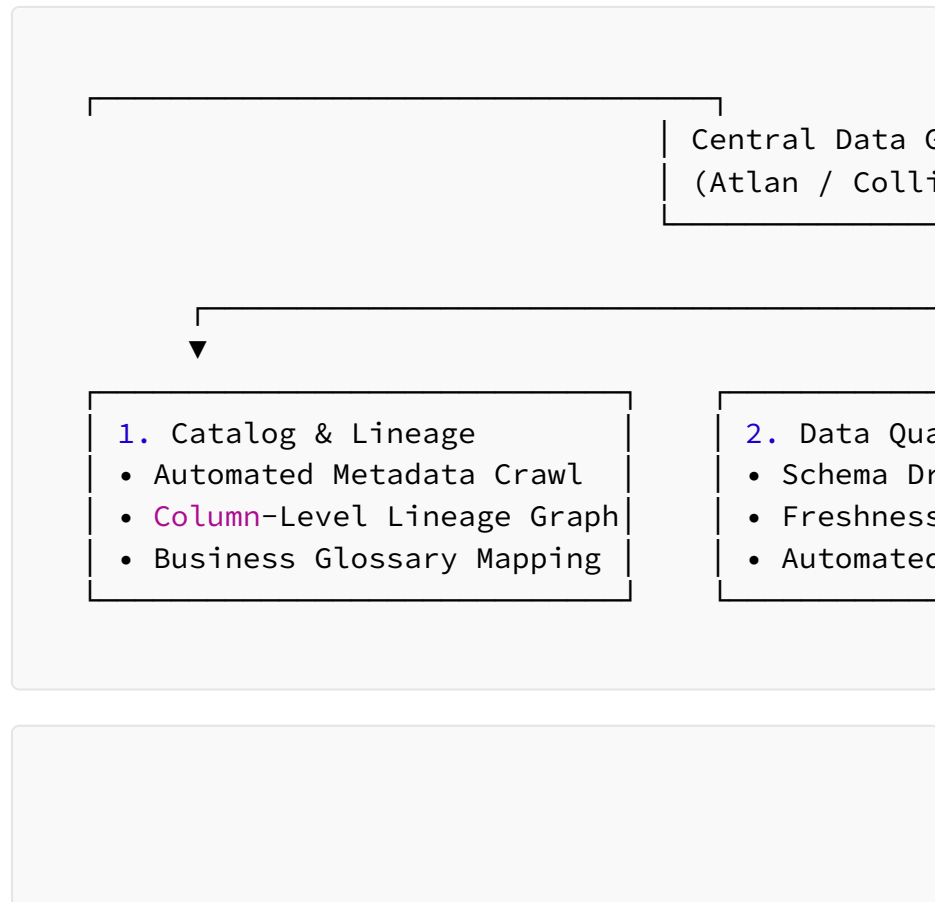


- **Schema Governance:** Centralized Schema Registry (Confluent/AWS Glue) utilizing Avro/Protobuf formats. CI/CD pipelines break builds on non-backward-compatible schema changes.
-
- **Topic Design & Partitioning:** Event topics partitioned deterministically by domain keys (e.g., `store_id`, `customer_id`) to ensure strict in-order processing per entity.
-
- **Resilience & Dead-Letter Queues (DLQ):** Unparseable payloads or unhandled consumer exceptions route to a isolated DLQ topic with exponential backoff retries and alert triggers.
-
- **Observability & Reconciliation:** End-to-end telemetry monitoring consumer group lag, payload sizes, and continuous automated source-to-target record count reconciliations.
-

6. Enterprise Lakehouse Architecture

Layer	Technology	Primary Function	Storage Layer
ADLS	Decoupled, immutable object storage serving as the single target store.	Table Format	Delta Lake / Apache Iceberg
Bronze Layer	Raw Storage	Append-only landing zone maintaining exact source data payloads.	Conformed Storage
Silver Layer	Cleansed, deduplicated, enriched datasets with standardized schemas.	Gold Layer	Dimensional Storage
Highly aggregated business data models (Star schemas/Data Marts) optimized for BI.	Compute Engines	Databricks, EMR, Trino, Redshift	Decoupled, auto-scaling compute clusters spun up on-demand for processing or querying.
Catalog & Control	Unity Catalog / AWS Glue	Centralized metadata registry, data lineage tracker, and access control engine.	

7. Data Governance Platform Architecture



- **Data Catalog & Lineage:** Automated metadata crawlers scan pipelines, databases, and BI tools to construct end-to-end, column-level data lineage graphs and business glossaries.
-
- **Data Quality & Observability:** Automated tests (dbt assertions / Great Expectations / Monte Carlo) validating data freshness, volume anomalies, null fractions, and schema mutations before downstream consumption.
-
- **Access Control & Privacy:** Policy-driven authorization engines (Immuta, Privacera, or AWS Lake Formation) enforcing Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC) with dynamic PII masking and column/row-level filtering.

-
- **Lifecycle Management:** Automated storage lifecycle policies managing data retention, cold storage archiving (Glacier), and regulatory compliance (GDPR/CCPA “Right to be Forgotten” deletes).
-

Below is the structure I would use for all seven. It is aligned with AWS’s current modern-data guidance: S3 as a scalable storage foundation, Glue Data Catalog and Lake Formation for metadata/governance, and purpose-built services such as Athena, Redshift, EMR, Kinesis/MSK, and AI/ML depending on the workload. [AWS Documentation](#)

The L6/L7 framework to memorize

Before answering **any** system-design question, say:

“Before I design the solution, I’d like to clarify the business objective, scale, latency, security requirements, availability requirements, and primary consumers.”

Then think:

R → I → S → P → G → C → O

Requirements → Ingestion → Storage → Processing → Governance → Consumption → Operations

That one framework can carry you through almost every data architecture question.

. . .

1. Design a Modern Data Platform

Step 1 — Clarify requirements

Ask:

“What business outcomes are we solving for?”

Then:

- Data volume today and 3-year growth?
- Batch, streaming, or both?
- Structured, semi-structured, unstructured?
- Number of source systems?

- Dashboard latency: seconds, minutes, hours?
- Data science/AI requirements?
- PII or regulated data?
- Global or single region?
- Existing Hadoop/Databricks/Snowflake/warehouse?
- RPO/RTO?
- Expected users and concurrency?

Suppose interviewer says:

Retail company, POS + e-commerce + ERP + CRM, 50 TB/day, some near-real-time requirements, BI + ML + GenAI, sensitive customer data.

Now design.

. . .

Step 2 — Sources

Sources could include:

- POS
- SAP/ERP

- CRM
- E-commerce
- Mobile
- SaaS
- Files
- IoT
- Third-party data

. . .

Step 3 — Ingestion

I separate ingestion by pattern.

Database CDC

Use something like AWS DMS where appropriate.

Why?

“I don’t want analytics pipelines repeatedly performing full database extracts.”

|

CDC captures changes incrementally.

Streaming

Use:

- Kinesis Data Streams for AWS-native streaming
- Amazon MSK if Kafka compatibility/ecosystem is important.

MSK is AWS's managed Apache Kafka offering, while MSK Serverless removes cluster-capacity management for suitable workloads. [AWS Documentation](#)

SaaS

AppFlow/connectors where appropriate.

Files

S3/API/managed transfer mechanisms depending on source.

. . .

Step 4 — Storage

I would generally establish **Amazon S3** as the **durable data foundation**.

Organize logically:

Raw → Validated → Curated

or

Bronze → Silver → Gold

Raw:

- immutable source data

Validated:

- standardized
- deduplicated
- quality checked

Curated:

- business-ready datasets/data products

Use efficient formats such as **Parquet**.

For lakehouse requirements, consider **Apache Iceberg**.

Important phrase:

“I separate storage from compute so different processing engines can operate against the same governed data without creating unnecessary copies.”



AWS's modern-data guidance similarly centers an S3-based data lake with governance and purpose-built analytics around it. [AWS Documentation](#)

. . .

Step 5 — Processing

Choose based on workload.

Glue

For managed/serverless ETL and relatively standardized transformations.

EMR

When I need:

- deep Spark tuning

- large-scale open-source workloads
- more runtime control
- existing Spark/Hadoop ecosystem

Lambda

Small event-driven transformations—not my default for massive Spark workloads.

. . .

Step 6 — Metadata & governance

Use:

AWS Glue Data Catalog

for centralized metadata.

AWS Lake Formation

for fine-grained data access.

Lake Formation can enforce row-, column-, and cell-level access across integrated AWS analytics services. [AWS Documentation](#)

. . .

Step 7 — Consumption

Different consumers get appropriate engines.

Athena

For serverless ad-hoc SQL directly against S3.

Redshift

For:

- enterprise warehouse workloads
- high-concurrency BI
- complex analytics
- predictable performance

EMR/Spark

For data engineering and advanced processing.

SageMaker / Bedrock

For ML and GenAI.

. . .

Step 8 — Security

Mention this without being asked:

- IAM least privilege
- KMS encryption
- private connectivity/VPC endpoints where required
- Lake Formation permissions
- Secrets Manager
- CloudTrail audit
- PII classification
- environment/account separation

That gets architect points.

. . .

Step 9 — Reliability

Discuss:

- idempotent pipelines
- retries
- DLQ where relevant
- checkpointing
- reconciliation
- backfills
- data-quality gates
- lineage
- disaster recovery

. . .

Step 10 — Cost

Talk about:

- S3 lifecycle
- Parquet/compression
- partitioning
- autoscaling/serverless where suitable
- shutting down idle compute
- Redshift workload sizing
- avoiding unnecessary copies

- FinOps tagging/chargeback

. . .

2-minute interview answer

“I would first clarify business outcomes, data volume, latency, consumers, governance and availability requirements.

At a high level, I would use S3 as the durable data foundation and support both batch and streaming ingestion. Database changes could arrive through CDC, while real-time events could use Kinesis or MSK.

I’d organize data into raw, validated and curated layers, generally using columnar formats and Iceberg where transactional lakehouse capabilities are required.

Glue or EMR would perform transformation depending on workload complexity. Glue Data Catalog provides metadata, while Lake Formation provides centralized fine-grained governance.

For consumption, Athena supports ad-hoc serverless querying, Redshift supports high-performance enterprise analytics, and curated datasets can also support ML and GenAI.

Across the architecture I’d design security, observability, data quality, lineage, recovery and cost optimization from day one.

Most importantly, I wouldn't choose every AWS service automatically. I'd select each component based on latency, scale, operating complexity and business requirements."

That is a strong L6/L7 answer.

. . .

2. Design Customer 360

First question:

"What business decisions will Customer 360 improve?"

Examples:

- personalization
- loyalty
- churn
- campaign targeting

- recommendations
- customer service

AWS's Customer Data Platform guidance explicitly covers ingestion, identity resolution, segmentation, analytics and activation across data such as POS, CRM, web/mobile and contact-center sources. [AWS Documentation](#)

Sources

Think:

POS + E-commerce + Loyalty + CRM + Mobile + Marketing + Contact Center

. . .

Ingestion

Batch:

- transaction history
- CRM
- loyalty

Streaming:

- clickstream

- mobile events
- real-time purchases

Could use Kinesis or MSK for real-time ingestion. [AWS Documentation](#)

. . .

Data foundation

Land source data in S3.

Then:

Raw customer records
↓
Clean/standardize
↓
Deduplicate
↓
Identity resolution
↓
Golden customer profile

. . .

Hardest part: Identity Resolution

Mention this prominently.

Customer can appear as:

- prem@gmail.com
- +66...
- loyalty ID 12345
- device ABC
- CRM ID XYZ

Need to determine which records represent the same person.

Use:

- deterministic matching
- probabilistic matching
- AWS Entity Resolution where appropriate

AWS provides Entity Resolution specifically for matching, linking and unifying records, and AWS guidance recommends cleaning/standardizing PII before matching. [AWS Documentation](#)

. . .

Golden profile

Create something like:

Customer ID

Name

Contact

Loyalty status

Lifetime value

Last purchase

Preferred categories

Channel preferences

Consent

Segments

But don't overwrite every source.

Maintain lineage back to source.

. . .

Governance

Very important because this is PII.

Mention:

- consent
- purpose limitation
- masking
- encryption
- retention
- right-to-delete where applicable
- Lake Formation fine-grained access
- Macie for sensitive-data discovery

Macie can automate discovery and reporting of sensitive data in S3.

[AWS Documentation](#)

. . .

Activation

Customer 360 isn't successful until it drives action.

Curated profile can support:

- campaigns
- personalization
- contact center
- recommendation systems
- churn models
- BI

Strong line:

| “Customer 360 is not a database project. The outcome is activation.”

|

|

. . .

Interview answer

| “I would start by asking which business use cases need Customer 360

| —for example personalization, churn, campaign targeting or

customer service.

I would ingest customer data from POS, CRM, loyalty, e-commerce, mobile and marketing platforms into a governed S3 foundation.

The critical stage is data quality and identity resolution. I would standardize names, emails, phones and identifiers, then use deterministic or probabilistic matching, potentially with AWS Entity Resolution, to generate a persistent enterprise customer ID.

From that I would create a governed golden customer profile while preserving lineage back to the source systems.

I'd then expose curated customer data to Redshift or Athena for analytics and to downstream campaign, personalization and ML systems.

Since the platform contains PII, consent, encryption, retention, masking, least privilege and auditability would be first-class requirements.

Finally, I would measure success through business metrics such as campaign conversion, churn reduction, customer lifetime value or improved service—not simply number of records unified.”

Excellent answer.

. . .

3. Design a Document Processing Platform

This is one of your strongest areas.

Requirements

Ask:

- Document types?
- PDF/image/email?
- Documents/day?
- Max size?
- Languages?
- Required fields?
- Accuracy target?
- Human review?
- ERP integration?
- Audit requirements?
- Processing SLA?

Suppose:

500,000 invoices/POs per day.

. . .

Architecture

User/vendor

↓

Upload/API

↓

S3

↓

EventBridge

↓

Workflow orchestration

↓

Document classification

↓

Textract

↓

Normalize/interpret

↓

Business validation

↓

Confidence decision

↓

High confidence → downstream

Low confidence → human review

AWS's current Intelligent Document Processing guidance similarly uses S3 events, EventBridge, Lambda, Textract, Bedrock-based processing, validation and observability. [AWS Documentation](#)

. . .

Extraction

Texttract:

Extract:

- supplier
- invoice #
- PO #
- tax
- amount
- dates
- line items
- tables

. . .

Bedrock

Explain carefully:

| “I wouldn’t allow an LLM to directly approve a financial transaction.”

|

|

Bedrock can help:

- classify document
- normalize unusual labels
- infer semantic field mappings
- explain exceptions

But final transaction passes:

schema validation + deterministic business rules + confidence threshold.

Excellent L7 statement.

. . .

Business rules

Examples:

- Does PO exist?
- Does supplier match?
- PO amount vs invoice?
- Duplicate invoice?
- Supported currency?
- Tax calculation?
- Goods receipt available?

. . .

Human-in-the-loop

High confidence + rules pass:

→ straight-through processing.

Low confidence/rule failure:

→ exception queue/human review.

Say:

“The confidence threshold should be driven by business risk, not engineering convenience.”

Very strong.

. . .

Storage

Original document:

S3

Structured extracted result:

S3 / database depending on access pattern.

Workflow state:
DynamoDB may fit.

. . .

Integration

After validation, publish a business event such as:

InvoiceValidated

Then:

- ERP
- Finance
- warehouse
- analytics

consume independently.

This creates loose coupling.

. . .

Failure handling

Ask yourself:

Textextract failure?

→ retry

Repeated failure?

→ exception/DLQ

ERP down?

→ retain event / retry without reprocessing entire document.

Duplicate upload?

→ idempotency key/document fingerprint.

. . .

Monitoring

Technical:

- latency
- errors
- retry rate
- workflow failure

Business:

- extraction accuracy
- human-review %
- straight-through-processing %
- cost/document
- turnaround time

L7 line:

“I monitor business KPIs alongside infrastructure KPIs.”

. . .

4. Design a Real-Time Analytics Platform

Start with latency.

Ask:

“When you say real time, do you mean 100 milliseconds, five seconds, or five minutes?”

That’s an architect question.

Suppose:

Retail wants inventory and sales dashboards within 5–10 seconds.

. . .

Architecture

POS / e-commerce / IoT

↓

Kinesis or MSK

↓

Flink

↓

Real-time destinations

S3 historical lake

. . .

Streaming backbone

Kinesis if:

- AWS-native
- managed
- simple operational model

MSK if:

- Kafka ecosystem
- existing Kafka skills
- portability
- Kafka clients/connectors already used

Don't say one is universally better.

. . .

Processing

Managed Service for Apache Flink can perform streaming:

- joins
- windows
- aggregations
- enrichment
- stateful processing
- real-time metrics

AWS documents Flink as its managed service for processing streaming data and building real-time metrics and dashboards. [AWS Documentation](#)

. . .

Serving

Depending on workload:

Redshift

Operational/business BI.

Redshift currently supports direct streaming ingestion from Kinesis and MSK into materialized views, avoiding an S3 staging step where that pattern is appropriate. [AWS Documentation](#)

OpenSearch

Search/log/operational exploration.

DynamoDB

Key-based operational serving.

And always:

S3

Historical/replay/archive.

. . .

Important architecture pattern

Don't build:

Stream → Dashboard only.

Build:

Hot path + cold path

Hot:
seconds

Cold:
durable history/reprocessing.

. . .

Failure considerations

Discuss:

- replay
- checkpoint
- late events
- out-of-order events
- duplicates

- poison messages
- consumer lag
- retention

This differentiates senior candidates.

. . .

5. Design a Streaming Architecture

This question is broader than real-time analytics.

The interviewer is testing whether you understand **distributed event systems**.

Clarify

Ask:

- throughput?
- number producers?
- event size?
- ordering?
- retention?
- replay?
- latency?

- delivery semantics?
- number of consumers?
- regional requirements?

. . .

Architecture

Producers

↓

Kinesis/MSK

↓

Topics/streams

↓

Consumer groups

↓

Processing

↓

Independent destinations

Consumers might include:

- fraud
- warehouse
- notifications
- ML
- search
- data lake

. . .

Partitioning

Very important.

Say:

| “Partition key determines both scalability and ordering.”
|
|
|

For orders:

partition by customer/order/store depending on required ordering.

Avoid hot partitions.

. . .

Schema

Use contracts.

Event:

event_id

event_type

event_version

timestamp

source

payload

Support schema evolution.

. . .

Idempotency

Assume duplicates can happen.

Every event gets unique ID.

Consumers should detect/reject duplicate processing.

Excellent architecture point.

. . .

Retry & DLQ

Transient failure:

retry with backoff.

Persistent failure:

DLQ/quarantine.

Don't block entire stream because one message is bad.

. . .

Backpressure

If producer > consumer:

- scale consumers
- increase stream capacity
- monitor lag
- apply throttling where necessary

. . .

Replay

Keep enough retention or archive to S3.

If downstream logic is wrong:

reprocess history.

. . .

Security

- TLS
- IAM
- encryption
- network isolation
- secrets where appropriate
- least privilege

. . .

Interview answer

“I design streaming systems around four things: throughput, ordering, replay and failure isolation.

Producers publish versioned events into Kinesis or MSK. I choose the partition key carefully because it determines scale and ordering.

Multiple consumers process events independently so analytics, fraud, notifications and storage are not tightly coupled.

Every consumer is designed to be idempotent. Transient failures are retried; unrecoverable events are quarantined rather than blocking the stream.

I monitor consumer lag, throughput, failed events and end-to-end latency, and retain events long enough to support replay.

The most important design decision isn't Kafka versus Kinesis. It's making sure the event contract, partitioning, failure model and operating model meet the business requirements."

Very strong.

. . .

6. Design a Lakehouse

First explain what it solves.

"A lakehouse combines the low-cost/open storage characteristics of a data lake with warehouse-like table management and analytics capabilities."

Don't make it simply:

S3 + Spark.

. . .

Architecture

Sources

↓

Ingestion

↓

S3

↓

Iceberg tables

↓

Glue Data Catalog

↓

Lake Formation

↓

Athena / Redshift / EMR

AWS's current lakehouse starter architecture uses S3, Glue catalog/ETL, data quality, Athena and Redshift; Lake Formation provides governance. [AWS Documentation](#)

. . .

Why Iceberg?

Explain concepts:

- ACID-style table transactions
- schema evolution
- partition evolution
- snapshot/time-travel capabilities

- multiple compatible engines

Don't say:

| "Iceberg is storage."

|

|

It is a **table format** over underlying data files.

. . .

Layers

Bronze

raw

Silver

clean + validated + conformed

Gold

business-ready products/aggregates

But at L7 add:

“I don’t create layers just because the medallion diagram says so.
Every additional copy must have a business or quality purpose.”

Nice senior point.

. . .

Compute

Different engines can operate on governed lakehouse data depending on workload.

- Athena: interactive/serverless SQL
- EMR: Spark/open-source processing
- Glue: ETL
- Redshift: warehouse/BI

Glue Data Catalog increasingly supports unified/federated metadata, including Apache Iceberg-compatible access patterns. [AWS Documentation](#)

. . .

Performance

Discuss:

- partition strategy
- avoid small files
- compaction
- Parquet
- statistics
- predicate pruning
- clustering/sorting where relevant

. . .

Governance

- central catalog
- row/column policies
- tagging

- lineage
- data ownership
- quality

. . .

When NOT to use lakehouse

This is a great follow-up.

“If the organization has a straightforward BI workload with highly structured data and no need for multiple engines or lake-scale storage, a conventional warehouse might be simpler.”

|

|

That’s architectural judgment.

. . .

7. Design a Data Governance Platform

This is commonly answered badly.

Don’t start with:

“Lake Formation.”

Start with:

“Governance is people + process + technology.”

That’s L7.

. . .

Step 1 — Governance operating model

Define:

- Data owner
- Data steward
- Data producer
- Data consumer
- Platform team

Example:

Finance owns revenue.

Data Engineering doesn't decide what revenue means.

. . .

Step 2 — Classification

Classify data:

Public

Internal

Confidential

Restricted/PII

. . .

Step 3 — Catalog

Use Glue Data Catalog for technical metadata.

Store:

- tables
- schemas
- location
- partitions

- classifications

AWS supports using Glue Data Catalog as a centralized metadata repository across S3, Redshift and federated sources. [AWS Documentation](#)

. . .

Step 4 — Business discovery

For enterprise self-service, something such as **Amazon DataZone** can support publishing, discovering and subscribing to governed data assets, including lineage and metadata capabilities. [AWS Documentation](#)

Think:

Amazon DataZone = discovery/data-product workflow

Glue Catalog = technical metadata

Lake Formation = authorization

That distinction is useful.

. . .

Step 5 — Access Control

Lake Formation.

Implement:

- database/table
- row
- column
- cell
- LF-tag based access

Example:

Marketing:

customer ID + segment

Finance:

customer ID + payment data

Analyst:

masked PII

Lake Formation supports fine-grained access across integrated analytics services. [AWS Documentation](#)

. . .

Step 6 — Sensitive-data discovery

Macie can identify sensitive data in S3. [AWS Documentation](#)

Then classify/tag accordingly.

. . .

Step 7 — Data quality

Every critical data product should have quality dimensions:

- completeness
- uniqueness
- validity
- freshness
- accuracy
- consistency

Example:

Customer ID NULL < 0.1%

Orders freshness < 15 min

Duplicates < threshold

Failures block promotion to curated layer or trigger alert.

. . .

Step 8 — Lineage

Need:

Source

→ transformation

→ table
→ semantic model
→ report

Why?

When CFO asks:

“Where did this revenue number come from?”

You need an answer.

. . .

Step 9 — Audit

Track:

Who accessed what?

Who changed permissions?

Who changed schema?

Who published data?

Use audit mechanisms such as CloudTrail plus platform-specific audit logs.

. . .

Step 10 — Lifecycle

Governance also means:

- retention
- archive
- deletion
- legal hold
- consent
- data residency

. . .

Strong interview answer

“I view data governance as an operating model rather than a single AWS service.

First, I define ownership: each critical data domain has a business owner and data steward.

Second, I establish classification and metadata. Glue Data Catalog provides the technical catalog, while enterprise discovery and data-product workflows can be provided through DataZone.

Third, I apply Lake Formation for centralized fine-grained access and use tag-based policies so governance scales across domains rather than creating thousands of manual grants.

Fourth, I introduce data-quality rules, lineage and auditability. Sensitive data can be discovered and classified, and access is logged.

Finally, I define lifecycle policies around retention, deletion and regulatory requirements.

My success metric would not be number of policies created. It would be whether trusted data can be discovered and accessed quickly by authorized users while sensitive data remains appropriately protected.”

. . .

The most important follow-up questions

For **every one** of these seven designs, expect the architect to interrupt you with questions like:

1. **Why did you choose that service?**
2. **What alternatives did you consider?**
3. **What happens if it fails?**
4. **What happens at 10x scale?**
5. **How do you secure it?**
6. **How do you monitor it?**
7. **How do you reduce cost?**
8. **How do you migrate to it?**
9. **How do you roll back?**
10. **What trade-off did you make?**

The best L7-style response is rarely:

“AWS recommends X.”

Instead:

“Given this requirement, I chose X because _____. The trade-off is _____.
If the requirement changed to _____, I would instead consider Y.”

That single pattern will make a major difference in your architecture interview.

Your 7 memory anchors

Modern platform: R-I-S-P-G-C-O

Customer 360: Identity resolution

Document AI: Confidence + business rules + human review

Real-time analytics: Hot + cold paths

Streaming: Partition + replay + idempotency

Lakehouse: S3 + Iceberg + multiple engines

Governance: People + metadata + policy + quality + audit

Do **not** try to memorize seven giant scripts. Memorize those seven anchors and the common architecture framework.

Section 4 — Migration (95%)

1. How would you migrate Oracle to AWS?
2. SQL Server migration?
3. Hadoop migration?
4. Data warehouse modernization?
5. Zero downtime migration?
6. CDC?
7. Rollback strategy?

. . .

A successful enterprise cloud migration balances schema conversion effort against operational downtime constraints. Below are the architectural blueprints, tooling strategies, and cutover protocols for migrating legacy databases, Hadoop clusters, and data warehouses to AWS.

1. Oracle & SQL Server Migrations

Select your migration strategy based on whether you are maintaining the same engine (homogeneous) or refactoring to an cloud-native engine (heterogeneous).

Migration Type	Source DB	Target AWS Architecture	Primary Tools
Homogeneous (Replatform)	Oracle / SQL Server	Amazon RDS or EC2 (Oracle / SQL Server)	Native Backup/Restore, RMAN, AWS DMS

SQL Server: Native `.bak` upload to S3 $\rightarrow$ RDS restore.

Oracle: RMAN / Data Pump export to S3 $\rightarrow$ RDS import.

Migration Type	Source DB	Target AWS Architecture	Primary Tools
Heterogeneous (Refactor)	Oracle / PostgreSQL / MySQL	Amazon Aurora	AWS SCT, AWS DMS
Heterogeneous (Refactor)	SQL Server	Amazon Aurora PostgreSQL (with Babelfish)	AWS SCT, Babelfish, AWS DMS

Convert PL/SQL DDL via **AWS Schema Conversion Tool (SCT)**. Perform initial load + continuous CDC via **AWS DMS**. Use **Babelfish for Aurora PostgreSQL** to understand T-SQL and TDS protocol natively, minimizing application code rewrites.

2. Hadoop Modernization (HDFS to AWS Data Lake)

Migrating from an on-premise Hadoop cluster (Cloudera / Hortonworks) to AWS requires decoupling compute from storage by

moving HDFS into Amazon S3.

Hadoop Component Mapping

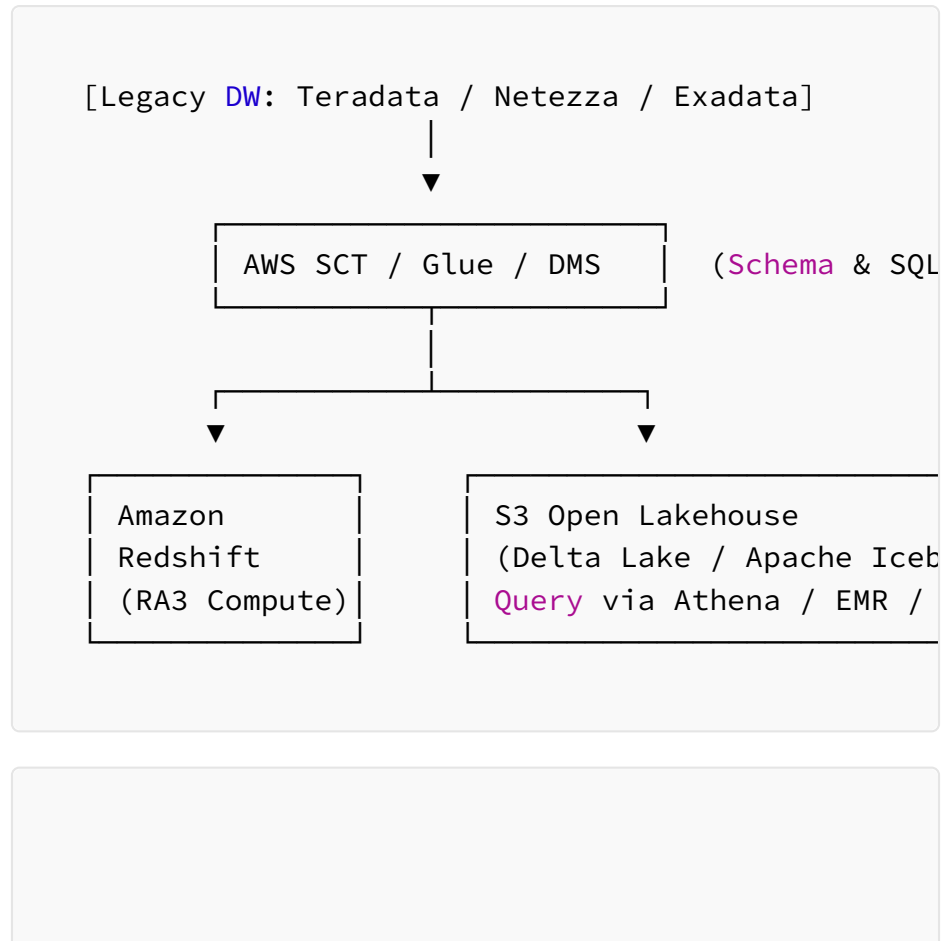
- **HDFS Storage** $\rightarrow$ **Amazon S3 / S3 Express One Zone** (Decoupled, durable object storage).
-
- **Hive Metastore** $\rightarrow$ **AWS Glue Data Catalog** (Centralized serverless metastore).
-
- **YARN / MapReduce / Spark** $\rightarrow$ **Amazon EMR / EMR Serverless or Databricks on AWS**.
-
- **HBase** $\rightarrow$ **Amazon DynamoDB or Amazon Keyspaces** (Cassandra-compatible).
-
- **Oozie / Airflow** $\rightarrow$ **Amazon Managed Workflows for Apache Airflow (MWAA)**.
-

Hadoop Data Transfer Strategy

1. **Historical Data:** Use `s3distcp` or **AWS DataSync** over **AWS Direct Connect** to copy HDFS data directly to S3 Bronze buckets in Parquet or ORC format.
- 2.
3. **Incremental Data:** Run periodic delta scripts or mirror incoming streaming pipelines (Kafka/MSK) to dual-write to both Hadoop and S3 during the transition window.
- 4.

3. Data Warehouse Modernization

Modernizing legacy warehouses (Teradata, Netezza, Exadata, Oracle DW) involves shifting from expensive, appliance-bound MPP hardware to an open AWS Lakehouse architecture.



- **Schema & Query Conversion:** Run AWS SCT to automatically convert enterprise warehouse DDLs, macros, views, and stored procedures into Amazon Redshift SQL or Spark SQL.
-
- **Storage Optimization:** Store cold historical data in S3 using open formats (**Apache Iceberg** or **Delta Lake**) and keep hot transactional reporting data in **Amazon Redshift (RA3 instances)** with managed storage.
-

- **Data Pipelines:** Replace legacy ETL tools (Informatica/Ab Initio) with **dbt + Databricks/EMR** or **AWS Glue**.
-

4. Zero-Downtime Migration & CDC Pattern

Achieving zero downtime requires continuous data replication to keep the target AWS database synchronized with the legacy source until cutover.

[Source DB] → [Transaction Redo Logs] → [CDC

[Target AWS DB] ← [Target Replication Task] ←

Execution Phases

- **Schema Pre-creation:** Deploy converted DDL (indexes, primary keys, schemas) to target AWS DB. Disable triggers and foreign key constraints on the target during initial load.
-
- **Full Load Phase:** AWS DMS or custom Spark jobs perform a bulk load of existing table data from source to target.
-
- **Continuous CDC Sync:** The CDC engine reads source redo/transaction logs (e.g., Oracle LogMiner, SQL Server

Change Tracking) and streams updates, inserts, and deletes to the target database.

-
- **Data Validation:** Execute continuous automated checksums (using AWS DMS Data Validation or custom hashing scripts) to ensure zero data drift between source and target.
-
- **Cutover Window (Near-Zero Downtime):**
- Pause incoming write traffic at the application layer (typically 5–15 minutes).
-
- Wait for replication lag to drop to **0 seconds**.
-
- Re-enable foreign keys and triggers on the target database.
-
- Switch DNS/connection endpoints to the AWS target and resume traffic.
-
-
-

5. Change Data Capture (CDC) Technical Implementations

Source Engine	CDC Mechanism	AWS Native Tool	Open-Source /
Enterprise Alternative	Oracle Log Miner	Binary Reader	AWS
DMS	Debezium	Oracle GoldenGate	SQL Server Change Data Capture (CDC) / Change Tracking
AWS DMS	Debezium	Qlik	Replicate
PostgreSQL	Logical Replication (pgoutput / wal2json)	AWS	

DMSDebeziumMySQLBinary Log (binlog) ROW modeAWS
DMSDebezium

6. Production Rollback Strategy & Cutover Safety

A migration plan is incomplete without an explicit fallback mechanism if issues arise post-cutover.

Reverse CDC (Bi-Directional Replication)

Set up a reverse replication task (AWS Target $\rightarrow$ Legacy Source) immediately after initial cutover. Any writes written to the new AWS database are continuously replicated back to the legacy system during the 24–72 hour burn-in period. If a critical bug is discovered, traffic can be pointed back to the legacy database with zero data loss.

Rollback Decision Tree & Safety Guardrails

- **Pre-Cutover Go/No-Go Gate:** If CDC lag is >0 or validation errors exceed 0.01% 30 minutes before cutover, immediately abort and remain on source.
-
- **Post-Cutover Fallback (Within Burn-in Period):**
- **Option A (With Reverse CDC):** Flip DNS/Route53 back to legacy source. System operates on legacy source without data loss.
-
- **Option B (Without Reverse CDC):** Export delta data written to AWS target using timestamp audit columns, re-ingest into legacy DB, then flip DNS back to source.
-

-
-
- **Point of No Return (PoNR):** Define an explicit deadline (e.g., 72 hours post-cutover) where reverse CDC is terminated, legacy hardware is decommissioned, and AWS becomes the permanent primary.
-

Yes. For **AWS Professional Services L6/L7**, migration questions are very important because they test more than AWS services. They want to see whether you can protect a customer's business while changing a critical platform.

Memorize one framework:

A → T → C → M → V → C → O

Assess → Target → Convert → Migrate → Validate → Cutover → Optimize

And always add **rollback**.

. . .

1. How would you migrate Oracle to AWS?

Don't immediately say "DMS."

First ask:

“Is the goal simply to move Oracle to AWS, or is the customer also trying to reduce Oracle licensing and modernize the database engine?”

That is important because there are two very different migrations.

Option A — Homogeneous

Oracle → RDS for Oracle

Lower application-change risk.

Option B — Heterogeneous

Oracle → Aurora PostgreSQL / RDS PostgreSQL

Higher transformation effort, but potentially removes Oracle dependency.

Current AWS DMS Schema Conversion supports Oracle conversions to targets including Aurora/RDS PostgreSQL, Aurora/RDS MySQL, and Redshift; DMS Schema Conversion handles schema/code

conversion while AWS DMS handles the data migration. [AWS Documentation](#)

Step 1 — Assessment

I inventory:

- Database size
- Growth rate
- Tables
- Stored procedures
- Packages
- Functions
- Triggers
- Views
- Materialized views
- LOBs
- Oracle-specific features
- Dependencies
- Peak TPS
- SLA
- RPO/RTO
- Network bandwidth

For heterogeneous migration, run a **schema-conversion assessment**.

It tells us:

what converts automatically
what requires remediation
what isn't supported.

AWS DMS Schema Conversion provides assessment reports specifically for this. [AWS Documentation](#)

. . .

Step 2 — Choose target

I don't choose PostgreSQL just because it's cheaper.

I compare:

RDS Oracle

- lowest change
- faster migration
- Oracle licensing remains

versus

Aurora PostgreSQL

- more modernization

- potential license savings
- greater schema/code conversion effort

I build a business case around:

| Cost + effort + risk + performance + long-term strategy.

|

|

. . .

Step 3 — Schema conversion

For Oracle → PostgreSQL:

Use DMS Schema Conversion.

Convert:

- Tables
- Datatypes
- Views
- Procedures
- Functions
- Triggers

Then manually remediate unsupported Oracle-specific logic.

. . .

Step 4 — Full load

Provision target.

Run initial migration:

Oracle → AWS DMS → Aurora/RDS

Existing application continues writing to Oracle.

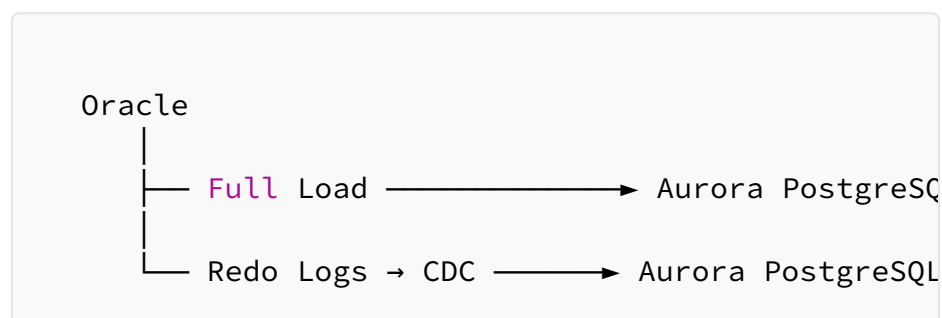
. . .

Step 5 — CDC

Now enable Change Data Capture.

DMS reads Oracle redo logs and continuously applies changes to the target while the existing application remains live. AWS DMS supports full-load-plus-CDC and reads Oracle changes using mechanisms such as LogMiner or Binary Reader. [AWS Documentation](#)

Architecture:



. . .

Step 6 — Validation

This is where a senior answer differs.

I validate:

Data

- Row counts
- Checksums
- Key totals
- NULL distribution
- Referential integrity

Application

- Queries
- APIs
- transactions

Business

For banking:

- account balances
- transaction totals

For retail:

- order totals
- inventory

AWS DMS also supports data validation that compares source and target data. [AWS Documentation](#)

. . .

Step 7 — Cutover

When CDC lag approaches an acceptable threshold:

1. Announce cutover.

2. Temporarily stop or restrict source writes.
3. Drain remaining CDC.
4. Validate critical tables.
5. Switch application connection.
6. Smoke-test target.
7. Monitor.

. . .

Step 8 — Rollback

Keep Oracle intact.

If validation fails **before target writes begin**:

| Point application back to Oracle.

|

|

|

If the new target has already accepted writes, rollback is more complicated. You need a preplanned reverse-sync/reconciliation mechanism rather than casually switching back.

Interview closing

“For me, database migration isn’t a DMS project. It’s a business-continuity project. DMS is one mechanism; assessment, validation, cutover and rollback determine whether the migration is actually successful.”

Excellent L7 line.

. . .

2. SQL Server migration

Use almost the same framework.

First ask:

“Are we rehosting SQL Server or modernizing away from SQL Server?”

Possible targets:

SQL Server → RDS SQL Server

or

SQL Server → Aurora PostgreSQL / RDS PostgreSQL/MySQL

DMS Schema Conversion currently supports SQL Server conversion paths to Aurora/RDS PostgreSQL and MySQL-family targets. [AWS Documentation](#)

. . .

Assessment

Inspect:

- T-SQL
- Stored procedures
- SQL Agent jobs
- SSIS
- linked servers
- CLR
- triggers
- indexes
- dependencies
- Windows authentication
- HA/DR
- licensing

This matters because migrating a SQL Server database is not the same as migrating the SQL Server ecosystem.

Great interview statement:

| “The hidden migration complexity is often outside the tables.”

|

|

. . .

Homogeneous

SQL Server

↓

RDS for SQL Server

Lower code-change risk.

. . .

Heterogeneous

SQL Server

↓

Aurora PostgreSQL

Need:

- schema conversion
- T-SQL remediation
- application compatibility testing

. . .

Data movement

Full load + CDC.

AWS DMS can capture SQL Server changes through its engine-specific transaction-log mechanisms. [AWS Documentation](#)

. . .

Testing

Compare:

- SQL results
- execution plans
- latency
- transactions
- indexes
- connection behavior

And don't assume indexes from SQL Server should simply be reproduced one-to-one.

Optimize for the target engine.

Strong closing

“I separate migration from modernization. First preserve correctness and business continuity; once stable, optimize the workload for the capabilities of the target database.”

. . .

3. Hadoop migration to AWS

This is a great one for you because of your DBS background.

Interviewer:

“How would you migrate an on-prem Hadoop platform to AWS?”

Start:

“First I’d clarify whether we’re simply moving Hadoop or modernizing the data architecture. I normally would not recreate an identical permanent HDFS architecture unless there is a strong workload reason.”

Excellent.

. . .

Existing environment

Typical:

Sources



HDFS



Hive / Spark / MapReduce



Reports

Target:

Sources



Amazon S3



Glue / EMR / Spark



Iceberg



Athena / Redshift / ML

The major architectural improvement:

Decouple storage and compute.

AWS guidance notes that EMR can use S3-backed storage so data persists independently of compute clusters; HDFS can still make sense for temporary hot/iterative workloads. [AWS Documentation](#)

. . .

Step 1 — Assess

Inventory:

- HDFS data volume
- file formats
- Hive tables
- Spark
- MapReduce
- Oozie
- Sqoop
- Kafka
- custom JARs
- dependencies
- SLAs
- security
- Kerberos
- workloads/users

Classify:

Retire / Retain / Replatform / Refactor

Don't migrate dead jobs.

L7 thinking:

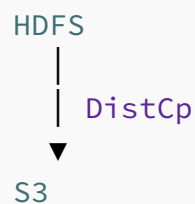
“Migration is an opportunity to remove technical debt, not reproduce it.”

. . .

Step 2 — Move data

For large online Hadoop migration, DistCp can move HDFS data to S3; AWS also documents offline or other transfer approaches depending on scale/network constraints. [AWS Documentation](#)

Example:



For extremely large datasets/network constraints, evaluate transfer alternatives.

. . .

Step 3 — Metadata

Hive Metastore

↓

Glue Data Catalog

where appropriate.

AWS also documents Hadoop/EMR migration patterns that move Hive metadata to Glue Data Catalog. [AWS Documentation](#)

. . .

Step 4 — Workloads

Map:

Hive → Athena/EMR

Spark → EMR/Glue

HDFS → S3

Oozie → managed orchestration

Kafka → MSK/Kinesis where appropriate

But don't blindly convert everything.

Step 5 — Parallel run

Old Hadoop

and

AWS

run simultaneously.

Compare:

- row counts
- aggregates
- runtime
- SLA
- output files
- BI results

. . .

Step 6 — Cutover by wave

Don't big-bang 1,000 jobs.

Wave 1:

low-risk jobs

Wave 2:
medium

Wave 3:
business-critical

. . .

Great closing

“The biggest mistake in Hadoop migration is treating it as an infrastructure migration. I’d use it to separate storage from compute, simplify operations, and retire unnecessary legacy workloads.”

Very L6/L7.

. . .

4. Data warehouse modernization

Suppose:

Teradata / Oracle Exadata / SQL Server DW

↓

AWS

First question:

“Why is the customer modernizing?”

Possible drivers:

- licensing
- scale
- performance
- AI readiness
- slow provisioning
- operational complexity

Then:

Assess

Inventory:

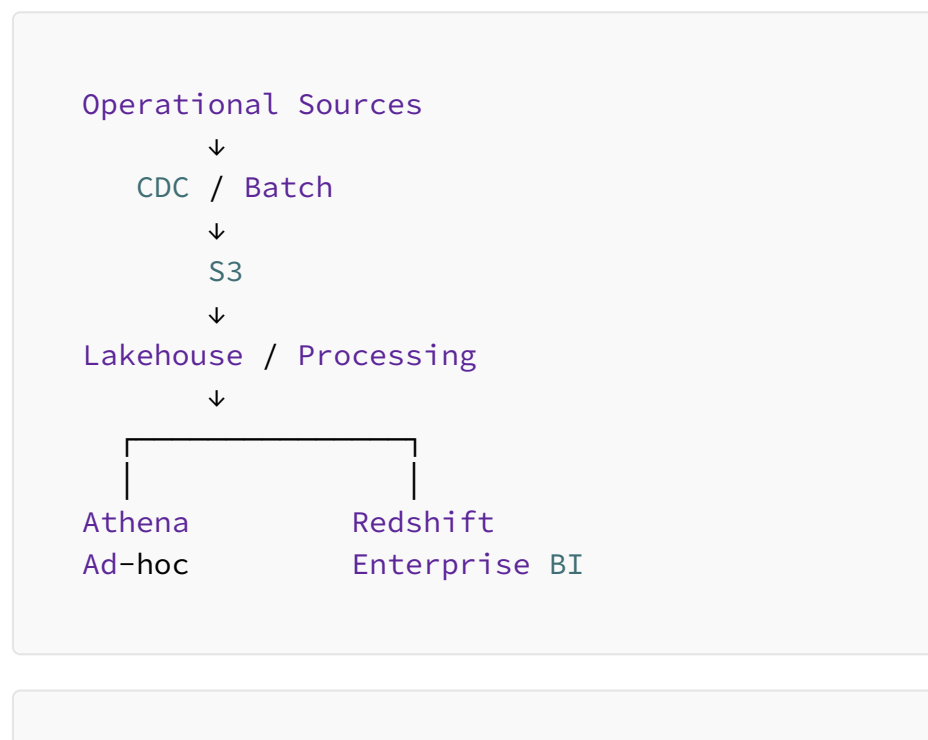
- data volume
- schemas

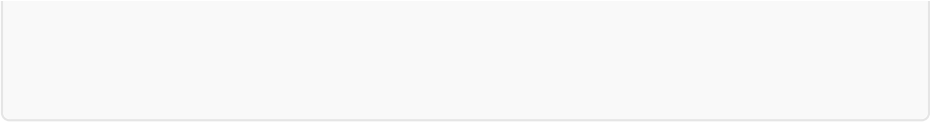
- ETL
- stored procedures
- BI tools
- workload concurrency
- query patterns
- SLAs
- upstream/downstream
- critical reports

. . .

Target architecture

Don't put everything into Redshift automatically.





Use workload placement.

. . .

Migration approach

For large warehouses:

wave-based migration.

AWS guidance explicitly discusses one-step, two-step, and wave-based Redshift migration strategies, with wave-based migration reducing complexity and risk for large MPP warehouses. [AWS Documentation](#)

I prefer:

Wave 0

Platform foundation

Wave 1

Medium-complexity domain

Wave 2

Additional business domains

Wave 3

Most critical workloads

Why not easiest first?

Sometimes a trivial workload proves almost nothing.

Choose something:

meaningful enough to validate architecture, but not business-critical
enough to create unacceptable risk.

Excellent consulting answer.

. . .

ETL modernization

Don't copy old ETL 1:1.

Evaluate:

- redundant transformations
- duplicate tables
- stale reports
- unused pipelines

Potentially move some transformation closer to lakehouse/warehouse.

. . .

Validation

Technical:

- counts
- reconciliation
- query results
- performance

Business:

Finance:

revenue/margin

Retail:

sales/inventory

Executive dashboards must reconcile.

. . .

Cutover

Run old/new platforms in parallel.

Then retire old workloads wave-by-wave.

L7 closing

“My objective isn’t to reproduce the legacy warehouse on AWS. It’s to preserve business semantics while simplifying the architecture and changing the operating model.”

. . .

5. Zero-downtime migration

Be careful with this phrase.

I would actually tell the interviewer:

“I would first clarify whether the requirement is truly zero downtime or near-zero downtime. For many database migrations, we can achieve near-zero application downtime using full load plus CDC, but I don’t promise literal zero without understanding transaction consistency and application behavior.”

That’s mature.

AWS DMS full-load-plus-CDC lets the target load historical data while source changes continue to be captured and applied. AWS also notes that CDC latency varies and doesn’t have a real-time replication SLA. [AWS Documentation](#)

. . .

Pattern

Phase 1

Source stays live.

```
App
↓
Oracle
↓
DMS full load
↓
Target
```

. . .

Phase 2

Start CDC.

```
Application → Source
                |
                └─ transaction logs
                    ↓
                  DMS
```

↓
Target

Users continue working.

. . .

Phase 3

Parallel validation.

Check:

- counts
- transactions
- queries
- performance

. . .

Phase 4

Cutover window

Temporarily pause writes.

Maybe seconds/minutes depending on workload.

Allow:

CDC lag → zero/acceptable

Then:

source

↓

target

Update application endpoints.

. . .

Phase 5

Monitor.

Don't immediately delete source.

. . .

6. Explain CDC

This needs a crisp answer.

“CDC stands for Change Data Capture. Instead of repeatedly copying the entire table, CDC captures inserts, updates and deletes that occur after the initial load and applies those changes to the target.”

Example:

Initial database:

100 TB.

You don't copy 100 TB every hour.

You initially copy 100 TB once.

Then:

```
graph TD; A[INSERT<br/>UPDATE<br/>DELETE] --> B[Transaction Log]; B --> C[DMS]; C --> D[Target]
```

INSERT
UPDATE
DELETE
↓
Transaction Log
↓
DMS
↓
Target

AWS DMS CDC uses source-engine transaction logs/native mechanisms; for example Oracle redo logs, SQL Server transaction-log mechanisms, MySQL binlogs, and PostgreSQL logical replication. [AWS Documentation](#)

. . .

Why CDC?

Low downtime

Application stays live during migration.

Network efficiency

Only changes move after full load.

Continuous synchronization

Target stays close to source.

. . .

Important concepts

CDC lag

Difference between source update and target apply.

Monitor:

- source latency
- target latency

Log retention

Very important.

If DMS is down for several hours but Oracle deletes required archive logs before DMS can consume them:

you can lose the migration stream.

So configure sufficient log retention.

. . .

CDC doesn't replace validation

Important:

“Replication success doesn't automatically mean business correctness.”

You still reconcile.

. . .

CDC interview answer

“I'd normally use full load plus CDC for a low-downtime database migration. The full load moves historical data while CDC reads transaction logs and captures ongoing inserts, updates and deletes. Once the initial load is complete and CDC lag is close to zero, I validate the target, briefly restrict writes, drain the remaining changes, and switch the application.”

Perfect.

. . .

7. Rollback Strategy

This one can separate senior candidates.

Don't answer:

| “We just switch DNS back.”

|

|

That can cause **data loss**.

. . .

Before migration

Create explicit go/no-go criteria.

Example:

Must have:

- 100% critical-table validation
- CDC lag under threshold
- performance tests pass
- security tests pass
- critical application journeys pass
- business owner sign-off

. . .

Before cutover

Maintain:

Source = system of record

Target receives replicated data.

Do not casually allow both systems to accept independent writes.

. . .

During cutover

I use checkpoints.

T-60

Validate replication.

T-30

Check CDC lag.

T-10

Stop/restrict source writes.

T-5

Drain CDC.

T-2

Final reconciliation.

To

Switch application.

. . .

After switch

Observe closely:

- errors
- latency
- business transactions
- application health
- database performance

Establish a rollback decision window.

. . .

Scenario A — Failure before target accepts writes

Easy.

Target not yet authoritative.

↓

Switch application back to source.

. . .

Scenario B — Target accepted new writes

Hard.

Now:

Source	Target
old data	+ new transactions

Simply switching back loses transactions.

Need one of:

- reverse replication
- transaction replay
- reconciliation
- dual-write mechanism, if designed correctly

- controlled manual recovery

This is why rollback must be designed **before** cutover.

Excellent line:

“Rollback is an architecture decision, not a runbook I write the night before migration.”

. . .

Keep old platform

Don't immediately terminate.

Have:

- agreed stabilization period
- backups
- source snapshots
- audit trail
- rollback window

Only decommission after acceptance.

. . .

Amazon L7 answer

“For every migration I define go/no-go criteria and rollback before implementation begins. Prior to cutover, the source remains authoritative while the target is synchronized through CDC. At cutover we restrict writes, drain replication, run final reconciliation, switch traffic, and monitor agreed business and technical metrics.

If failure occurs before writes reach the target, rollback is straightforward. If the target has accepted new writes, simply pointing traffic back would cause data loss, so I need a predesigned reverse synchronization or transaction-reconciliation strategy.

My principle is that I never perform a production cutover unless I know exactly how I will recover from it.”

That’s an excellent Senior ProServe answer.

. . .

Your migration memory card

Don't memorize seven answers.

Remember:

A-T-C-M-V-C-O-R

A—Assess

T—Target

C—Convert

M—Migrate

V—Validate

C—Cutover

O—Optimize

R—Rollback

Then remember the special concept for each:

QuestionAnchorOracleLicense vs modernizationSQL ServerT-SQL +
ecosystem dependenciesHadoopDecouple storage &
computeWarehouseWave-based modernizationZero downtimeFull
load + CDC + short cutoverCDCTransaction logs + lagRollbackWhat
happens to new writes?

If the architect goes deep, repeatedly use this senior pattern:

“I chose X because of requirement Y. The trade-off is Z. If the
requirement changed, I would consider another approach.”

That is far more important for L6/L7 than memorizing the names of AWS migration services.

Section 5 — AWS Services (90%)

1. Why S3?
2. Why Redshift?
3. Why Athena?
4. Glue vs EMR?
5. Lambda vs ECS?
6. EventBridge vs SQS?
7. DynamoDB vs Aurora?
8. Bedrock vs SageMaker?
9. Textract?
10. Step Functions?

. . .

Section 6 — Data Engineering (90%)

1. ETL vs ELT.

2. Batch vs Streaming.
3. Kafka vs Kinesis.
4. Data quality.
5. Data lineage.
6. Iceberg.
7. Parquet.
8. Partitioning.
9. CDC.
10. Medallion architecture.

. . .

Section 7 — Security (85%)

1. How do you secure S3?
2. IAM?
3. Encryption?
4. KMS?
5. Secrets Manager?
6. Multi-account strategy?
7. Least privilege?

. . .

Section 8 — Cost Optimization (85%)

1. Customer says AWS is expensive.
2. How do you optimize cost?
3. Reduce storage cost.
4. Optimize Spark jobs.
5. Serverless vs EC2.

. . .

Section 9 — AI (80%)

1. AI document processing.
2. RAG.
3. Bedrock.
4. LLM architecture.
5. Human-in-the-loop.
6. Confidence score.
7. Prompt engineering.

AWS L6/L7 GENAI INTERVIEW ANSWERS

Executive Opening (Use Before Any Answer)

“I approach GenAI solutions using a business-first and production-first mindset. My framework is Problem → Data → Model → Guardrails → Human Oversight → Measurement. Most GenAI projects fail not because the model is weak, but because of poor data quality, lack of governance, missing human review processes, and unclear success metrics.”

This sounds like AWS Principal Consultant / Senior Advisory Consultant level.

1. AI DOCUMENT PROCESSING

What is it?

AI Document Processing automates extracting information from documents such as:

- Invoices
- Receipts
- Contracts
- Purchase Orders
- KYC Documents
- Medical Records
- Insurance Claims
- Shipping Documents

Traditional OCR only extracts text.

Modern AI Document Processing understands:

- Context
- Relationships
- Meaning
- Tables
- Signatures
- Document Types

AWS Architecture

Plain Text

Customer Uploads PDF

2

3

|

4

▼

5

6

S3 Bucket

7

8

|

9

▼

10

11

Amazon Textract

12

13

|

14



15

16

Extract Text + Tables + Forms

17

18



19



20

21

Amazon Bedrock LLM

22

23



24



25

26

Classification

27

Validation

28

Summarization

29

30

|

31

▼

32

33

Confidence Scoring

34

35

|

36

▼

37

Human Review

38

(if confidence < threshold)

39

40

|

41

▼

42

43

ERP / CRM / Workflow

Show more lines

Example

Invoice Processing

Input:

Plain Text

1

Vendor: ABC Ltd

2

Amount: \$12,550

3

Due Date: 15/08/2026

Show more lines

AI Extracts:

JSON

1

{

2

“vendor”:”ABC Ltd”,

3

“amount”:12550,

4

“due_date”:”2026-08-15”

5

}

Show more lines

Then Bedrock validates:

Plain Text

1

Is the vendor approved?

2

Amount above approval limit?

3

Missing fields?

Show more lines

L7 Talk Track

|

|

“The business value of AI document processing isn’t OCR. The value comes from reducing manual review, accelerating workflow execution, and improving accuracy while maintaining governance.”

|

|

|

2. RAG (Retrieval Augmented Generation)

Biggest Interview Question

Why not fine-tune everything?

Fine-tuning changes model knowledge.

RAG changes context.

RAG is cheaper.

More secure.

More current.

Problem

LLM knows internet data.

It doesn't know:

- Your company policies
- Makro inventory
- CP Aextra promotions
- Internal SOPs

So hallucinations happen.

RAG Architecture

Plain Text

1

User Question

2

3

|

4

▼

5

6

Embedding Model

7

8

|

9

▼

10

11

Vector Database

12

13

|

14

▼

15

16

Relevant Documents Retrieved

17

18

|

19

▼

20

21

Prompt Construction

22

23

|

24

▼

25

26

LLM

27

28

|

29



30

31

Grounded Answer

Show more lines

Deep Dive

Step 1

Documents:

Plain Text

1

Contract.pdf

2

Policy.docx

3

Wiki Pages

4

Emails

Show more lines

Step 2

Chunking

Plain Text

1

1000 token chunks

Show more lines

Why?

LLMs cannot process huge documents efficiently.

Step 3

Embeddings

Convert text into vectors.

Plain Text

1

“return policy”

2

3

↓

4

5

[0.21,0.66,0.89...]

Show more lines

Step 4

Vector Search

Find nearest similar chunks.

Plain Text

1

Top K = 5

2

``

Show more lines

Step 5

Prompt Construction

Plain Text

1

Answer only using context.

2

3

Context:

4

[retrieved chunks]

5

6

Question:

7

User query

Show more lines

Production Challenges

Bad Chunking

Most common failure.

Context Overflow

Too many chunks.

Irrelevant Retrieval

Garbage retrieval.

Garbage answer.

Metadata Filtering

Must filter:

- Region
- Country
- Business Unit

L7 Answer

“The hard problem in RAG isn’t the LLM. The hard problem is retrieval quality. If retrieval quality increases from 60% to 90%, answer quality improves dramatically without changing the model.”

3. AMAZON BEDROCK

What is Bedrock?

Managed Generative AI Platform.

Like:

Plain Text

1

EC2 → Compute

2

3

Bedrock → Generative AI

Show more lines

Why Customers Want Bedrock

Avoid:

- Managing GPUs
- Managing models
- Managing infrastructure

Bedrock Components

Foundation Models

Examples:

- Claude

- Llama
- Amazon Nova
- Mistral

Knowledge Bases

Managed RAG

No need to build vector search manually.

Agents

Tool calling

Workflow orchestration

Guardrails

Prevent:

- PII leakage
- Toxic output
- Unsafe responses

Prompt Management

Version-controlled prompts.

Bedrock Enterprise Architecture

Plain Text

1

Application

2

3

|

4

▼

5

6

Amazon Bedrock Agent

7

8

|

9



10

11

Knowledge Base

12

13



14



15

16

Vector Store

17

18



19



20

21

Foundation Model

22

23

|

24

▼

25

26

Response

Show more lines

L7 Answer

|

|

|

|

|

“I position Bedrock as an enterprise GenAI control plane. Customers gain model flexibility, governance, security, and operational simplicity without being locked into a single model provider.”

4. LLM ARCHITECTURE

AWS loves this question.

Training Phase

Plain Text

1

Internet Data

2

3

|

4

5

▼

6

7

Tokenization

8

9

|

10

11

▼

12

13

Transformer Training

14

15

|

16

17

▼

18

19

Foundation Model

Show more lines

Inference Phase

Plain Text

1

Prompt

2

3

|

4

5

▼

6

7

Tokenization

8

9

|

10

11

▼

12

13

Transformer Layers

14

15

|

16

17

▼

18

19

Probability Prediction

20

21

|

22

23

▼

24

25

Next Token

Show more lines

Transformer Components

Attention

Most important concept.

Question:

Plain Text

1

Which prior words matter?

2

``

Show more lines

Example

Plain Text

1

The cat sat on the mat because it was tired.

Show more lines

What is “it”?

Attention discovers:

Plain Text

1

it → cat

2

``

Show more lines

Feed Forward Network

Learns patterns.

Positional Encoding

Maintains order.

Without it:

Plain Text

1

Dog bit man

2

3

Man bit dog

Show more lines

would appear identical.

L7 Interview Answer

“LLMs fundamentally perform probabilistic next-token prediction using transformer architectures. Business outcomes depend less on model size and more on data quality, retrieval strategy, governance, and workflow integration.”

5. HUMAN-IN-THE-LOOP (HITL)

Why?

No company should blindly trust AI.

Especially:

- Healthcare
- Legal
- Banking
- Retail Pricing
- Compliance

Example

Loan Approval

Wrong:

Plain Text

1

AI approves loan

Show more lines

Correct:

Plain Text

1

AI recommends

2

3

Human approves

Show more lines

Architecture

Plain Text

1

User Request

2

3

|

4

5

▼

6

7

LLM

8

9

|

10

11



12

13

Confidence Score

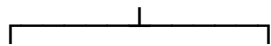
14

15



16

17



18

19

High Low

20

21



22

23

Auto Human

24

Approve Review

Show more lines

L7 Answer

“Human-in-the-loop isn’t a model weakness. It’s an enterprise governance requirement. High-risk decisions should always include human oversight regardless of model confidence.”

6. CONFIDENCE SCORE

AWS asks this often.

What Is It?

A numerical estimate of certainty.

Example:

Plain Text

1

Vendor Name = 98%

2

3

Invoice Total = 99%

4

5

Due Date = 47%

Show more lines

Sources of Confidence

OCR Confidence

Texttract confidence.

Retrieval Confidence

Vector similarity score.

Model Confidence

Token probability.

Business Rule Confidence

Validation checks.

Real Production Design

Plain Text

1

Score > 95%

2

Auto approve

3

4

90–95%

5

Sample review

6

7

<90%

8

Human review

9

..

Show more lines

Interview Insight

|

|

“Confidence score should never be viewed in isolation. The combination of model confidence, retrieval confidence, and business-rule validation provides more reliable decision making.”

|

|



7. PROMPT ENGINEERING

AWS almost always asks this.

Definition

Designing instructions that maximize LLM reliability.

Weak Prompt

Plain Text

1

Summarize this.

Show more lines

Better Prompt

Plain Text

1

You are a finance analyst.

2

3

Summarize this invoice.

4

5

Output:

6

- Vendor

7

- Amount

8

- Due Date

9

10

If missing data say UNKNOWN.

Show more lines

Production Framework

Role

Plain Text

1

You are a retail demand forecasting expert.

Show more lines

Task

Plain Text

1

Analyze sales trend.

2

Show more lines

Context

Plain Text

1

Store Sales Data

Show more lines

Constraints

Plain Text

1

Use only supplied information.

Show more lines

Output Format

JSON

1

{

2

“forecast”:...
}

3

```
}
```

Show more lines

Advanced Techniques

Few-Shot

Provide examples.

Chain-of-Thought

Break reasoning into steps.

Self-Critique

Model reviews answer.

RAG-Augmented Prompting

Inject retrieved context.

Agent Prompting

Enable tool usage.

L7/Principal Consultant Final Answer

“Successful enterprise GenAI systems are not built around a model. They are built around an architecture. That architecture includes document ingestion, retrieval, prompt orchestration, model inference, confidence scoring, human governance, and continuous feedback loops. The companies that succeed treat LLMs as one component of a larger business system rather than the entire solution.”

Memorize This One-Liner

Data → Retrieval → Prompt → LLM → Guardrails → Human Review → Feedback Loop → Business Value

This is the framework that works for almost every AWS Bedrock, GenAI, RAG, and AI Transformation interview question at the L6/L7 level.

Section 10 — Scalability (80%)

1. Handle 10x traffic.
2. High availability.
3. Disaster recovery.
4. Multi-region.
5. Retry strategy.
6. Dead Letter Queue.
7. Idempotency.

. . .

Section 11 — Consulting (95%)

These are very common in AWS Professional Services.

1. How do you run a discovery workshop?
2. How do you collect requirements?
3. How do you prioritize requirements?
4. What if the customer doesn't know what they want?
5. How do you convince stakeholders?
6. How do you manage scope?
7. What if the customer rejects your design?
8. How do you handle conflicting stakeholders?
9. How do you communicate with executives?
10. How do you explain technical concepts to non-technical leaders?

. . .

AWS L6/L7 Professional Services Interview Strategy

Best Format (Always Start Like This):

|

|

|

“I follow a structured consulting approach: Discover → Align → Prioritize → Execute → Measure Outcomes. My goal is not to sell technology. My goal is to solve business problems, align stakeholders, and create measurable business value.”

|

|

This opening immediately signals:

- Customer Obsession
- Ownership
- Dive Deep
- Earn Trust
- Deliver Results

Exactly what AWS L6/L7 interviewers want.

1. How do you run a discovery workshop?

L6 Answer

I start by understanding business outcomes before discussing technology.

My discovery workshop typically has four parts:

Business Vision

- What problem are we solving?

- Why now?
- What success looks like?

Current State Assessment

- Existing architecture
- Data landscape
- Process bottlenecks
- Organizational challenges

Future State Vision

- Desired capabilities
- User journeys
- Business KPIs

Prioritized Roadmap

1.

- Quick wins
- Medium-term initiatives
- Long-term transformation

I ensure every requirement can be traced to a measurable business outcome.”

L7 Answer

“I view discovery workshops as executive alignment exercises rather than technical sessions.

My primary objective is creating a shared definition of success across executives, business teams, operations, security, and engineering.

I typically leave the workshop with:

- Strategic business goals
- Success metrics
- Decision framework
- Risk register
- Transformation roadmap

The workshop is successful only if stakeholders leave aligned on priorities and investment decisions.”

2. How do you collect requirements?

L6

“I collect requirements from multiple sources:

- Executive interviews
- Business process walkthroughs
- User shadowing
- Architecture reviews

- Operational data

I separate requirements into:

- Business requirements
- Functional requirements
- Non-functional requirements
- Compliance requirements

I validate requirements back with stakeholders to ensure shared understanding.”

L7

“I focus less on what stakeholders ask for and more on what business outcomes they seek.

Customers often ask for capabilities. I translate those capabilities into measurable outcomes.

For example:

Customer Says: ‘We need real-time analytics.’

I ask: ‘What business decision cannot be made today because analytics arrive too late?’

That shifts the discussion from features to value.”

3. How do you prioritize requirements?

L6

“I prioritize using a business value versus implementation effort framework.

I evaluate:

- Business impact
- Customer impact
- Risk reduction
- Cost optimization
- Delivery complexity

High-value, low-effort items become immediate priorities.”

L7

“I prioritize based on enterprise impact.

I ask:

1. Does it drive revenue?
2. Does it reduce risk?
3. Does it improve customer experience?
4. Does it unlock future capabilities?

I avoid prioritizing based solely on stakeholder influence.

The roadmap should reflect organizational value, not organizational politics.”

4. What if the customer doesn't know what they want?

L6

“This is very common.

I use workshops, prototypes, architecture diagrams, and use-case discussions to help customers discover requirements.

I never ask:

‘What do you want?’

Instead I ask:

‘Walk me through your business process today.’

Pain points naturally emerge.”

L7

“Customers usually know their problems better than their solutions.

My role is helping them articulate the future state.

I focus on:

- Current pain
- Business goals
- Constraints

Then I present alternatives.

Customers don't hire consultants to record requirements. They hire us to clarify ambiguity."

5. How do you convince stakeholders?

L6

"I use data, prototypes, and business outcomes.

I avoid technical debates.

Instead of saying:

'This architecture is better.'

I say:

'This option reduces deployment time from 2 weeks to 2 hours.'

Results are more convincing than technology."

L7

“I don’t try to win arguments.

I build alignment.

I present:

- Options
- Tradeoffs
- Risks
- Costs
- Expected outcomes

I let stakeholders make informed decisions.

Trust is built through transparency.”

6. How do you manage scope?

L6

“I define scope upfront and document assumptions.

Whenever new requests emerge I categorize them as:

- In Scope
- Future Phase
- Out of Scope

Then I explain impacts on:

- Cost
- Timeline
- Resources

This keeps expectations realistic.”

L7

“Scope management is expectation management.

I continuously remind stakeholders:

‘Which business objective does this request support?’

If it doesn’t materially improve outcomes, it usually belongs in a future phase.

I protect delivery by protecting focus.”

7. What if the customer rejects your design?

L6

“I first seek to understand why.

Possible reasons:

- Technical concerns
- Operational concerns
- Budget concerns
- Organizational concerns

I review feedback objectively and adjust where appropriate.

The goal is solving the problem, not defending my design.”

L7

“I never become attached to solutions.

If a customer rejects my design, I ask:

‘What risk or concern is preventing adoption?’

Often the issue isn’t architecture. It’s organizational readiness.

I treat rejection as additional discovery.”

8. How do you handle conflicting stakeholders?

L6

“I listen to each stakeholder independently.

I identify:

- Goals
- Concerns
- Success metrics

Then I facilitate discussion around shared objectives.

The discussion moves from opinions to outcomes.”

L7

“Conflicting stakeholders usually indicate misaligned incentives.

I elevate discussions to enterprise goals.

For example:

Operations wants stability. Business wants speed.

Both ultimately want customer satisfaction.

I reframe conflicts around common objectives.”

9. How do you communicate with executives?

L6

“I keep communication concise.

Executives care about:

- Business outcomes
- Risks
- Investments
- Timelines

I avoid unnecessary technical details.”

L7

“I communicate in the language of business strategy.

Instead of:

‘We’ll implement a Lakehouse architecture.’

I say:

‘We’ll establish a unified data foundation that reduces reporting latency by 80% and improves decision making.’

Executives buy outcomes, not architecture diagrams.”

10. How do you explain technical concepts to non-technical leaders?

L6

“I use business analogies.

For example:

A data platform is like a supply chain.

Raw data enters, gets processed, quality checked, and delivered to consumers.

This makes complex systems understandable.”

L7

“I translate technology into business impact.

Instead of explaining AI models, I explain:

- Faster decisions
- Lower costs
- Better customer experiences
- Reduced risk

The best explanation is the one that leads to action.”

Ultimate L7 Closing Statement

When AWS asks any stakeholder/customer question, finish with:

|

|

|

|

|

“My role as a consultant is not simply delivering technology. My responsibility is aligning business goals, stakeholders, and technology investments to achieve measurable outcomes while minimizing risk. Success is not measured by deployment. Success is measured by customer business results.”

This sounds like a **Principal Consultant / L7 AWS Professional Services** answer and consistently maps to AWS Leadership Principles:

- Customer Obsession
- Ownership
- Earn Trust
- Dive Deep
- Think Big
- Bias for Action
- Deliver Results
- Have Backbone; Disagree and Commit

**Memorize this framework: Discover → Align → Prioritize → Execute
→ Measure Outcomes**

You can use it to answer ~80% of AWS Professional Services behavioral and consulting questions.

The 10 questions I would prepare first

If you only have limited time, focus on these:

1. Tell me about yourself.
2. Why AWS Professional Services?
3. Tell me about a difficult migration.
4. Tell me about earning trust without authority.
5. Design a modern data platform.

6. Design an AI document processing system.
7. How would you migrate a legacy data warehouse to AWS?
8. How do you handle data quality and governance?
9. What would you do if a customer disagreed with your architecture?
10. Tell me about your biggest technical achievement.

. . .

My prediction

Based on the interviewer being a **Sr. ProServe Cloud Architect**, I think the interview will likely be:

- **20%**—Leadership Principles (STAR stories)
- **60%**—Technical architecture and design
- **20%**—Consulting approach (customer discovery, trade-offs, stakeholder management)

So spend most of your preparation time practicing **designing systems and explaining your design decisions**, rather than memorizing AWS service names. A senior ProServe architect will care much more about *why* you made a design choice than whether you can list every AWS service.